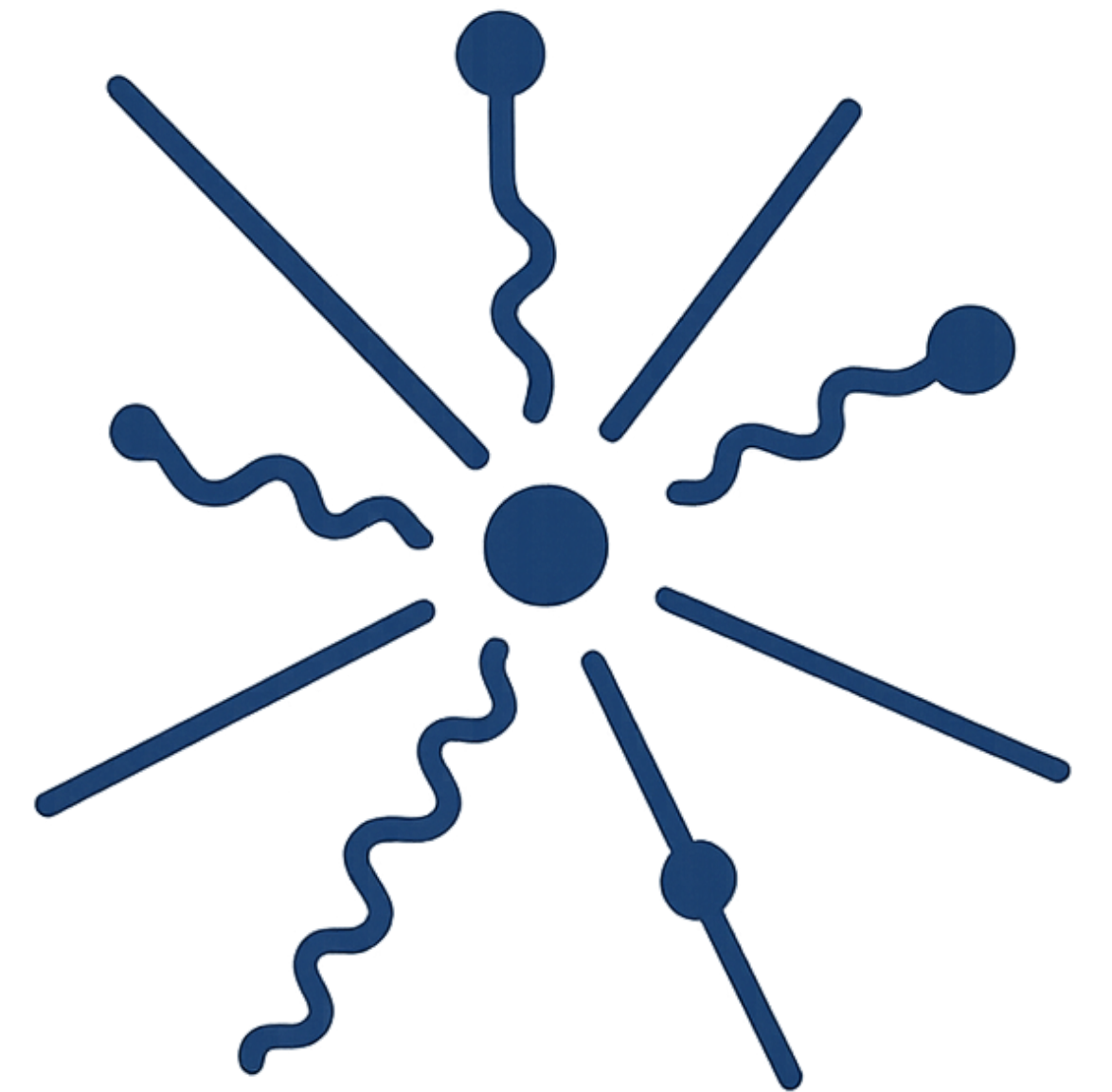
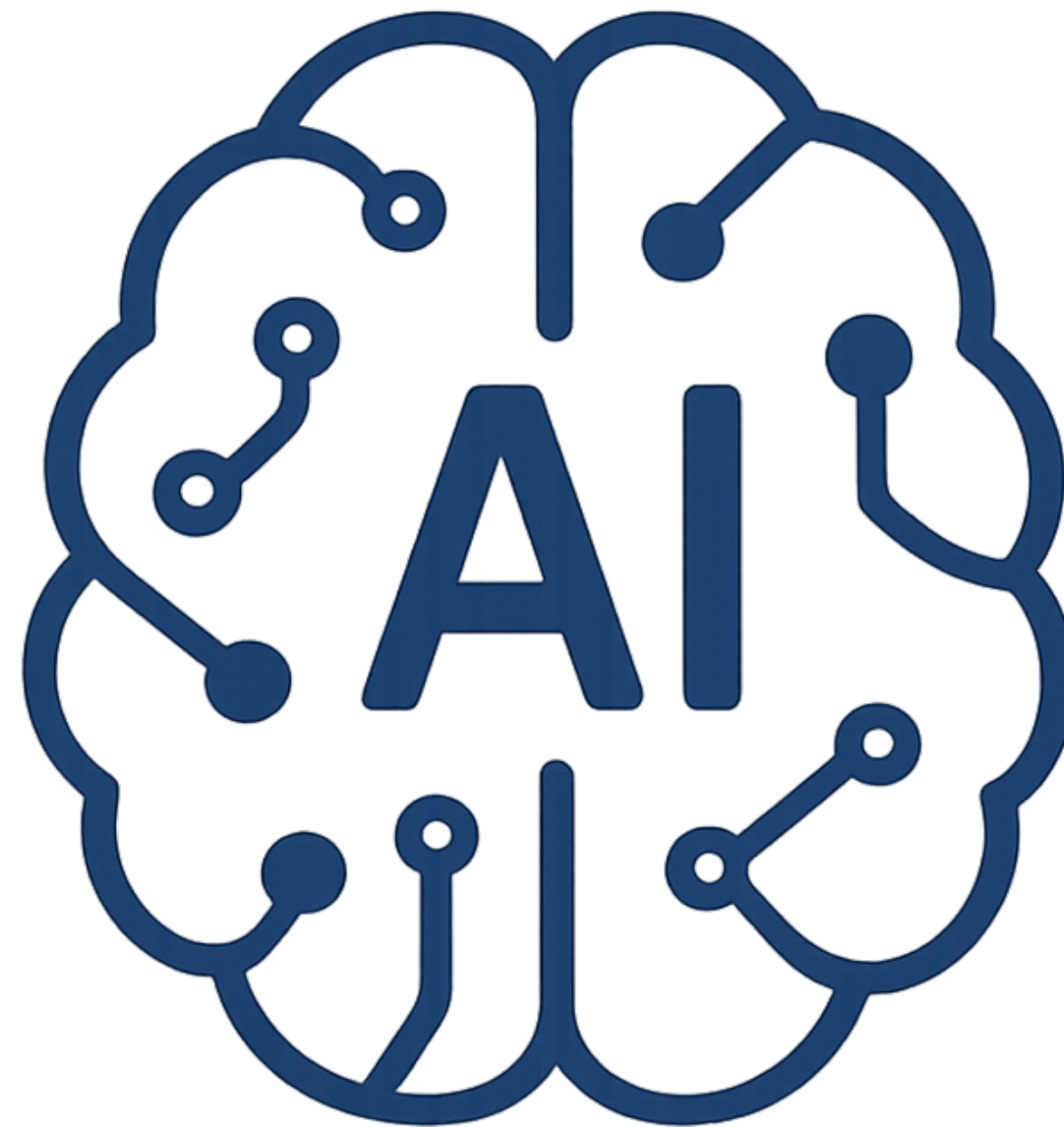
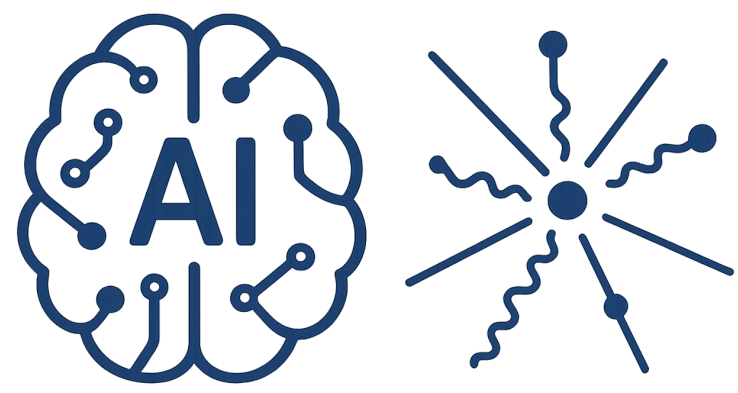


Introduction to

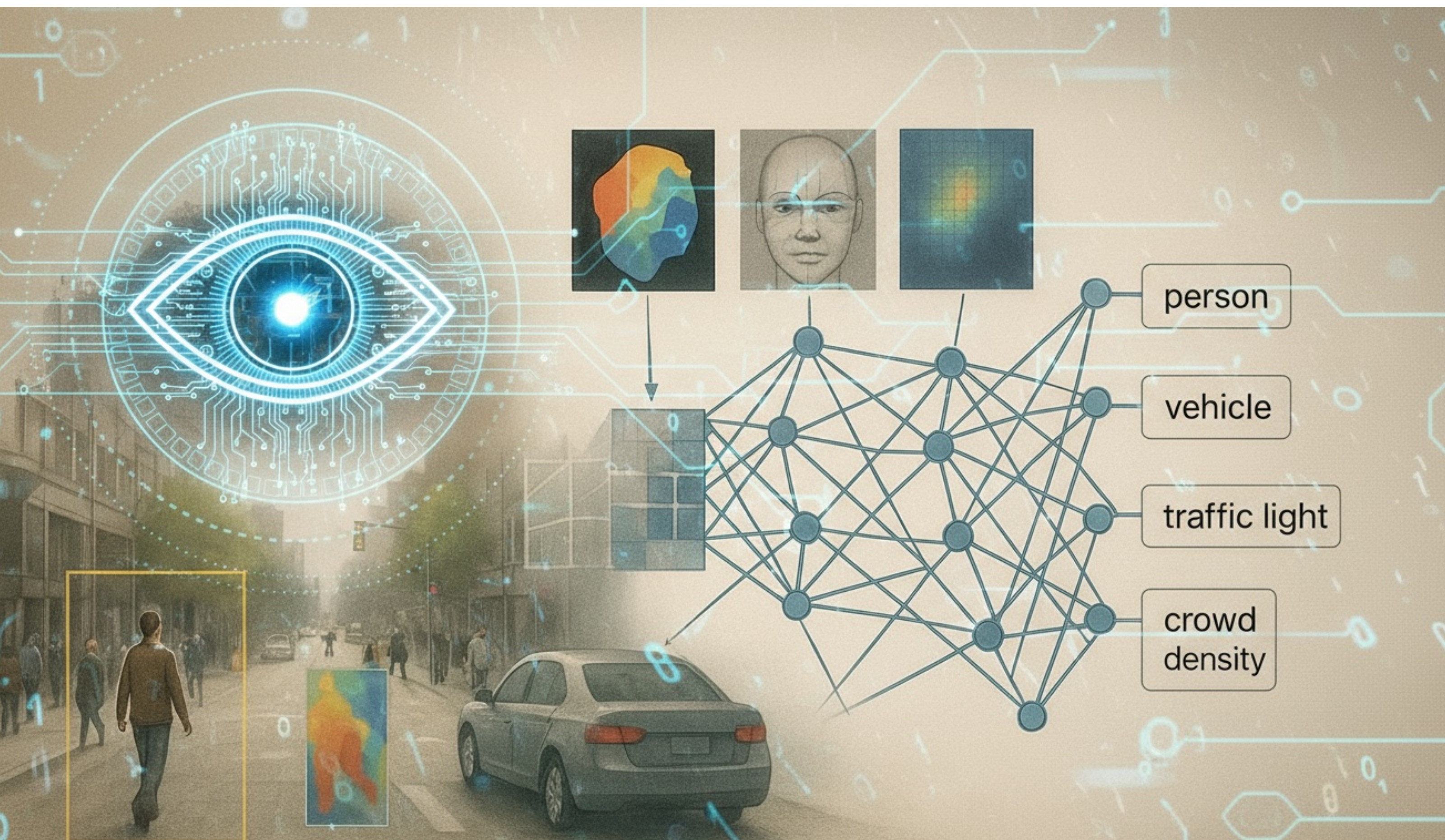
AI-Driven HEP

S. A. Fard - School of Physics (IPM)



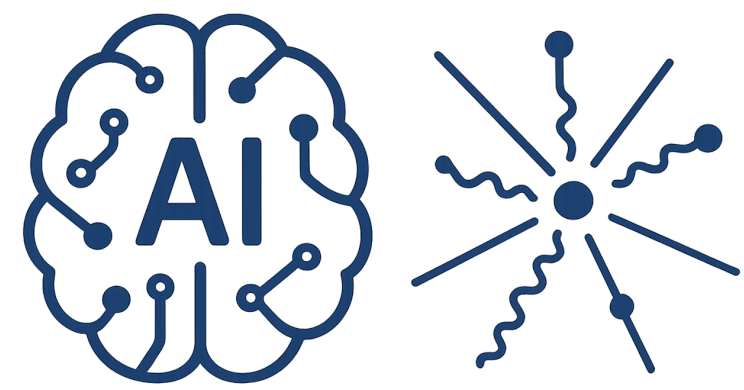


AI-Driven HEP 13



Session 13

Computer Vision II

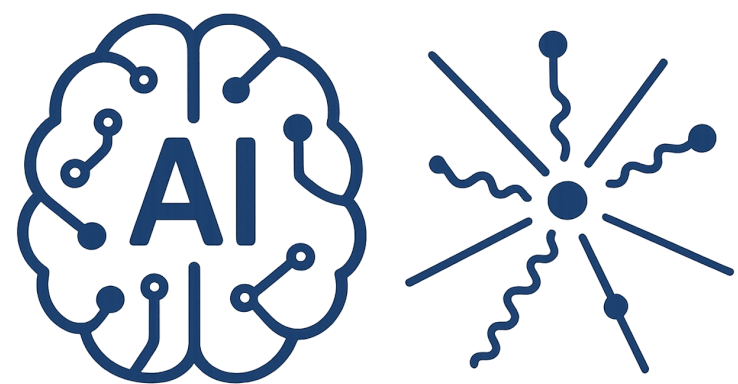


AI-Driven HEP 13

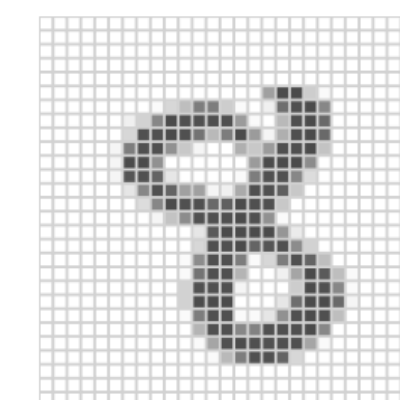
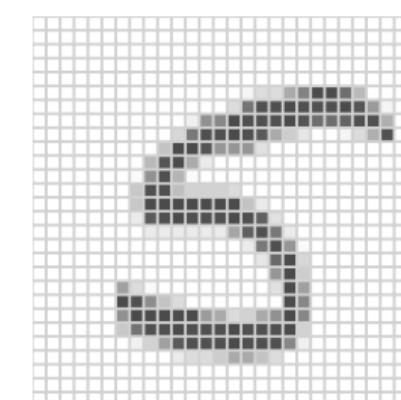
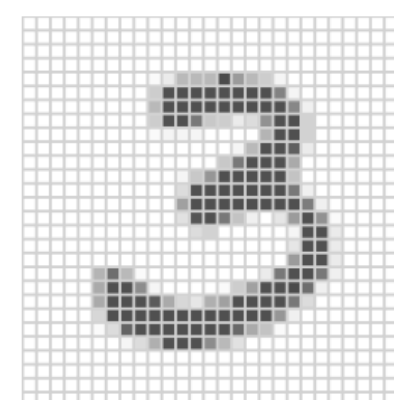
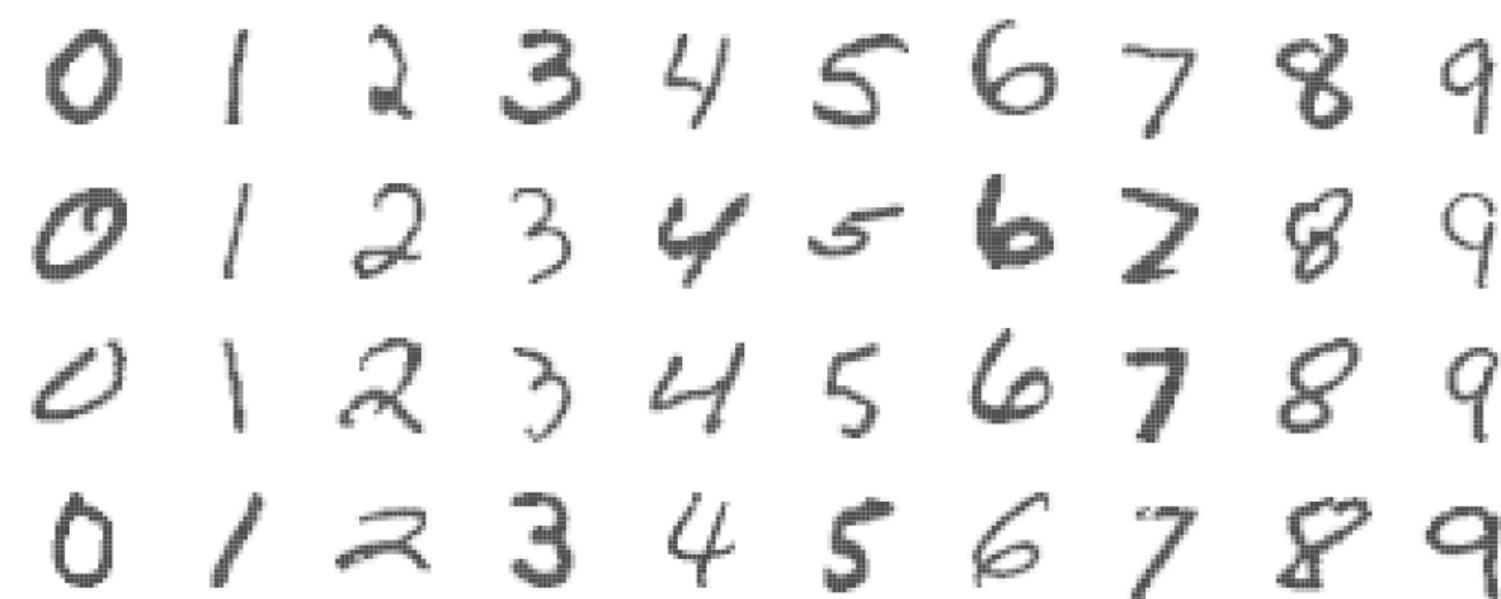
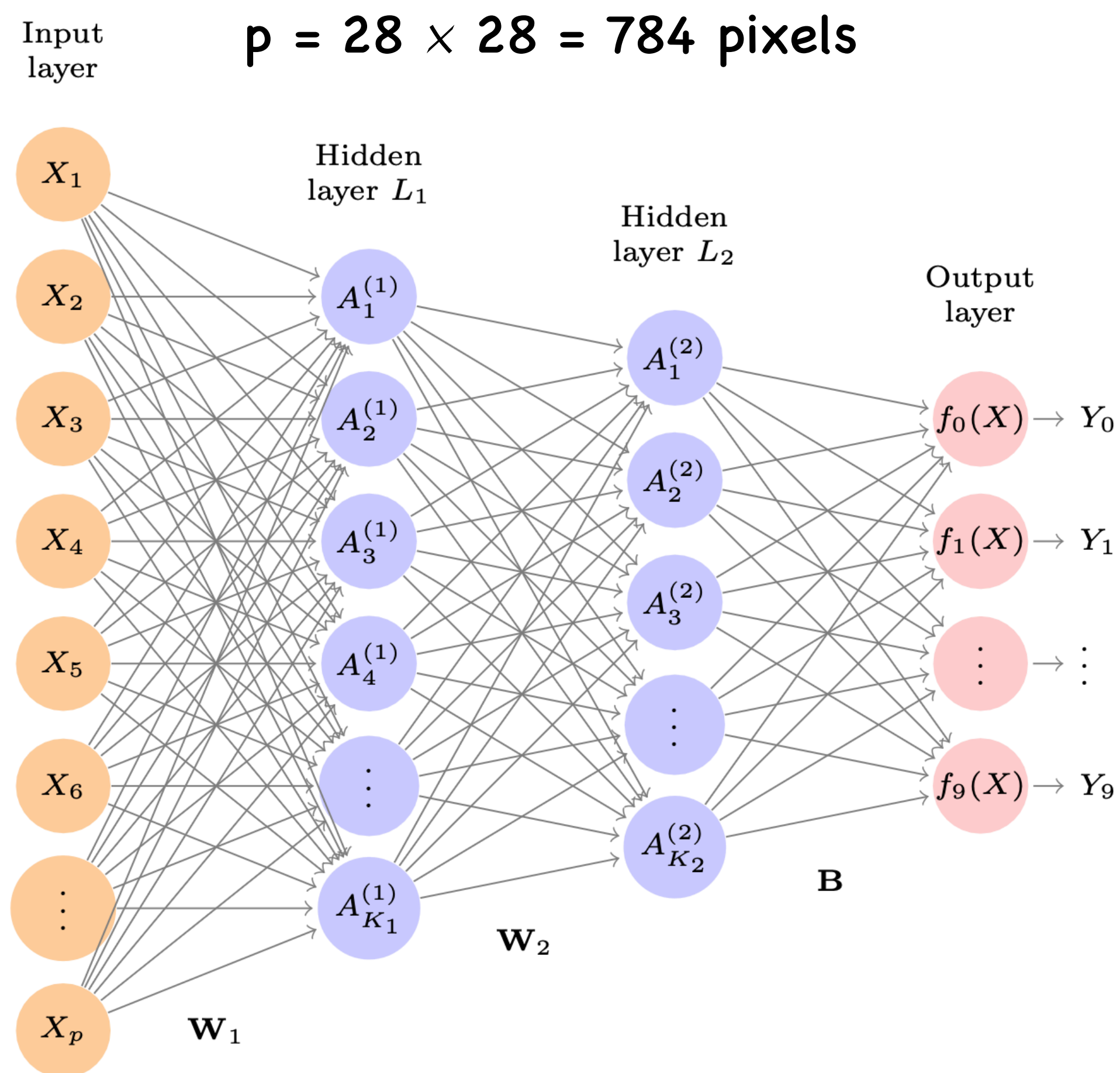
Reference

- [1] Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Springer Nature, 2023.
- [2] James, Gareth, et al. "Statistical learning." An introduction to statistical learning: With applications in Python. (2023)
- [3] Foundations of Computer Vision, <https://visionbook.mit.edu/>

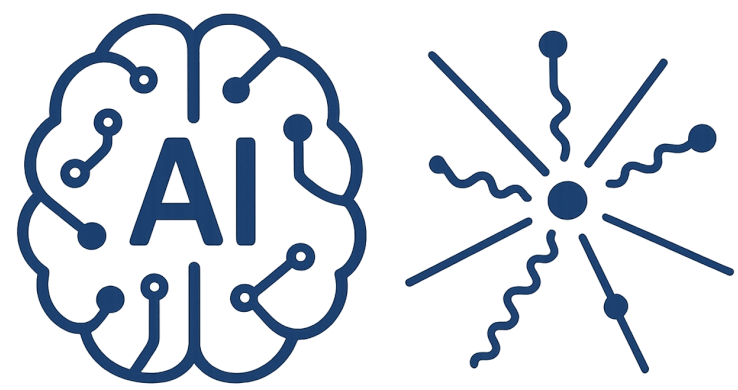
<https://physics.ipm.ac.ir/~ansarifard/>



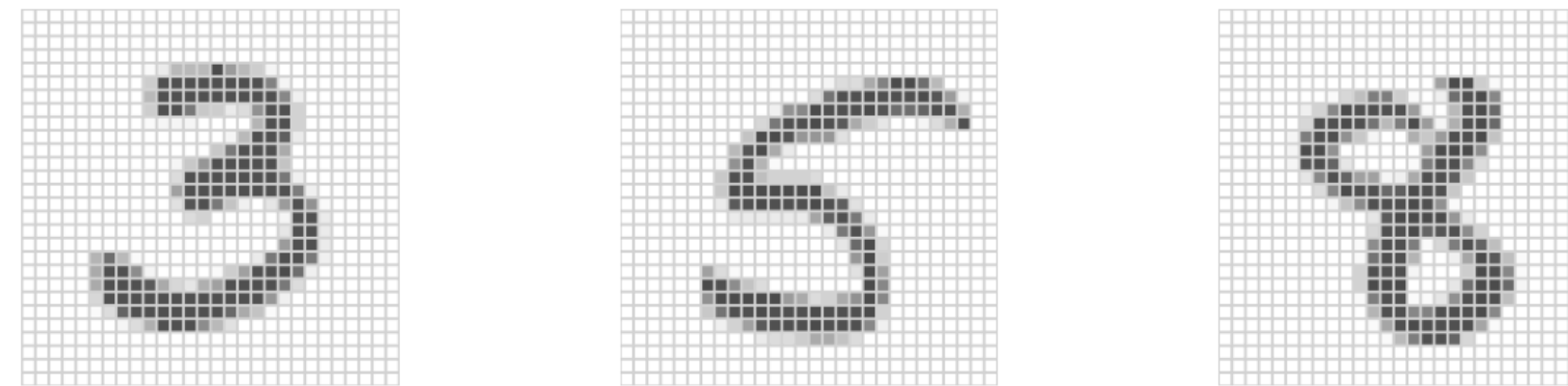
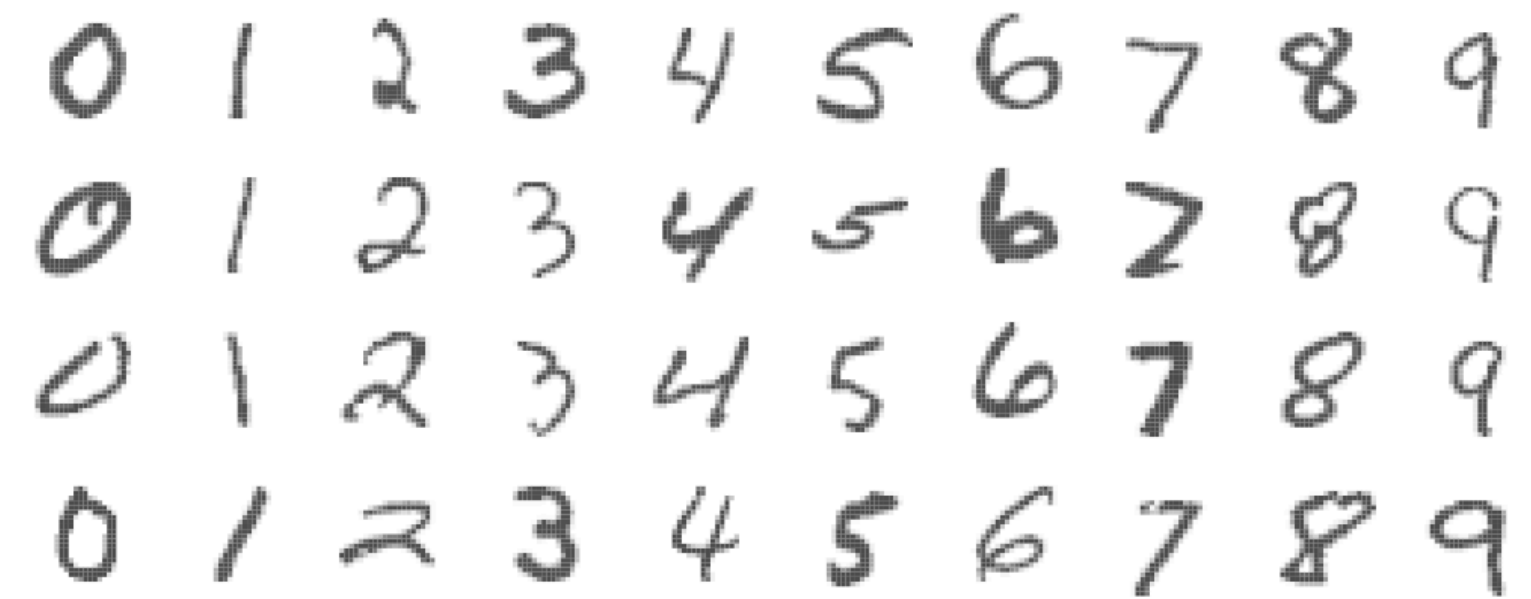
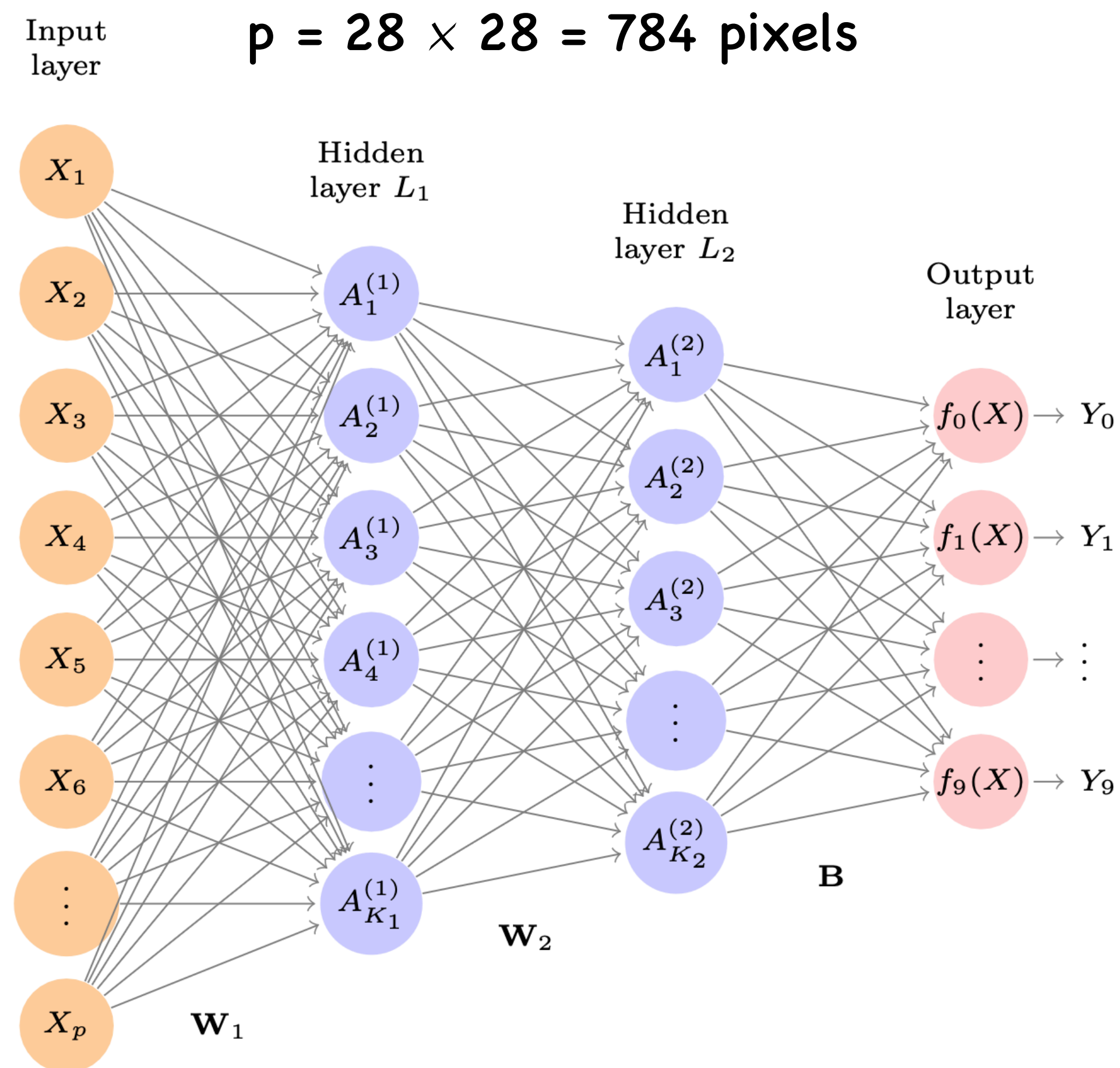
HandWrite Digit Classification



$\sim 235,000$ weights parameter



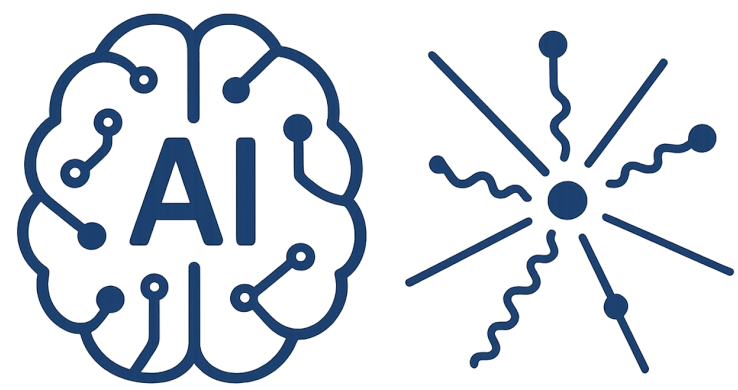
HandWrite Digit Classification



$\sim 235,000$ weights parameter

What does this number means ?

How we can choose it ?

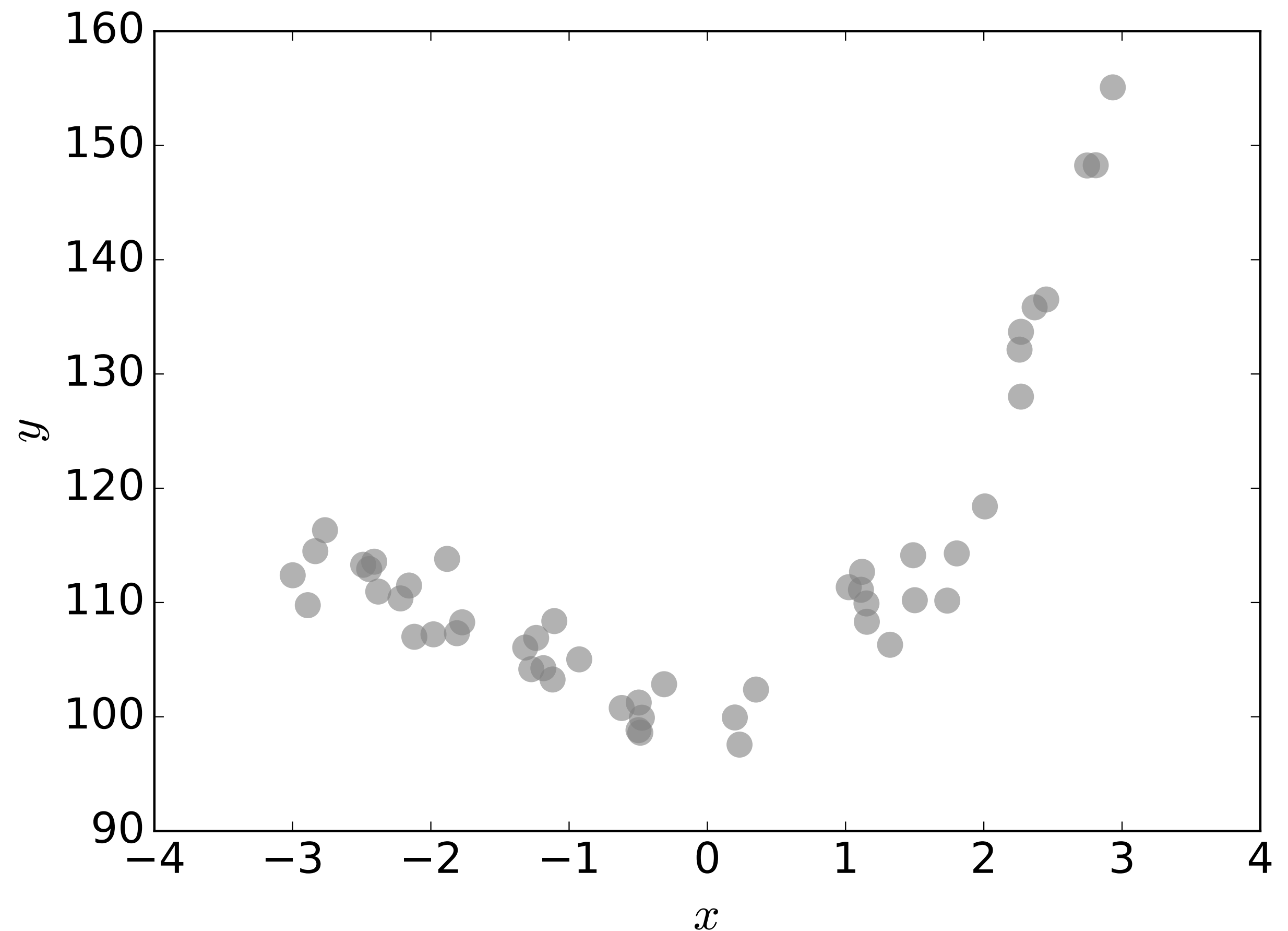


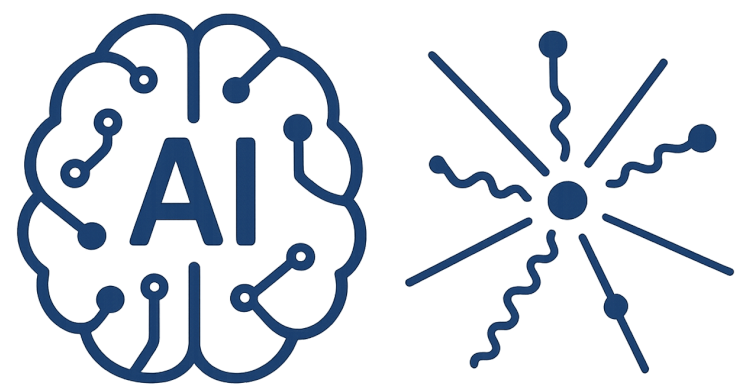
Parameters of the Model

Suppose we have a sample which is drawn from a population

Unstructured data such as intensity as a function of energy (spectrum)

x	y
x_1	y_1
x_2	y_2
x_3	y_3
x_n	y_n





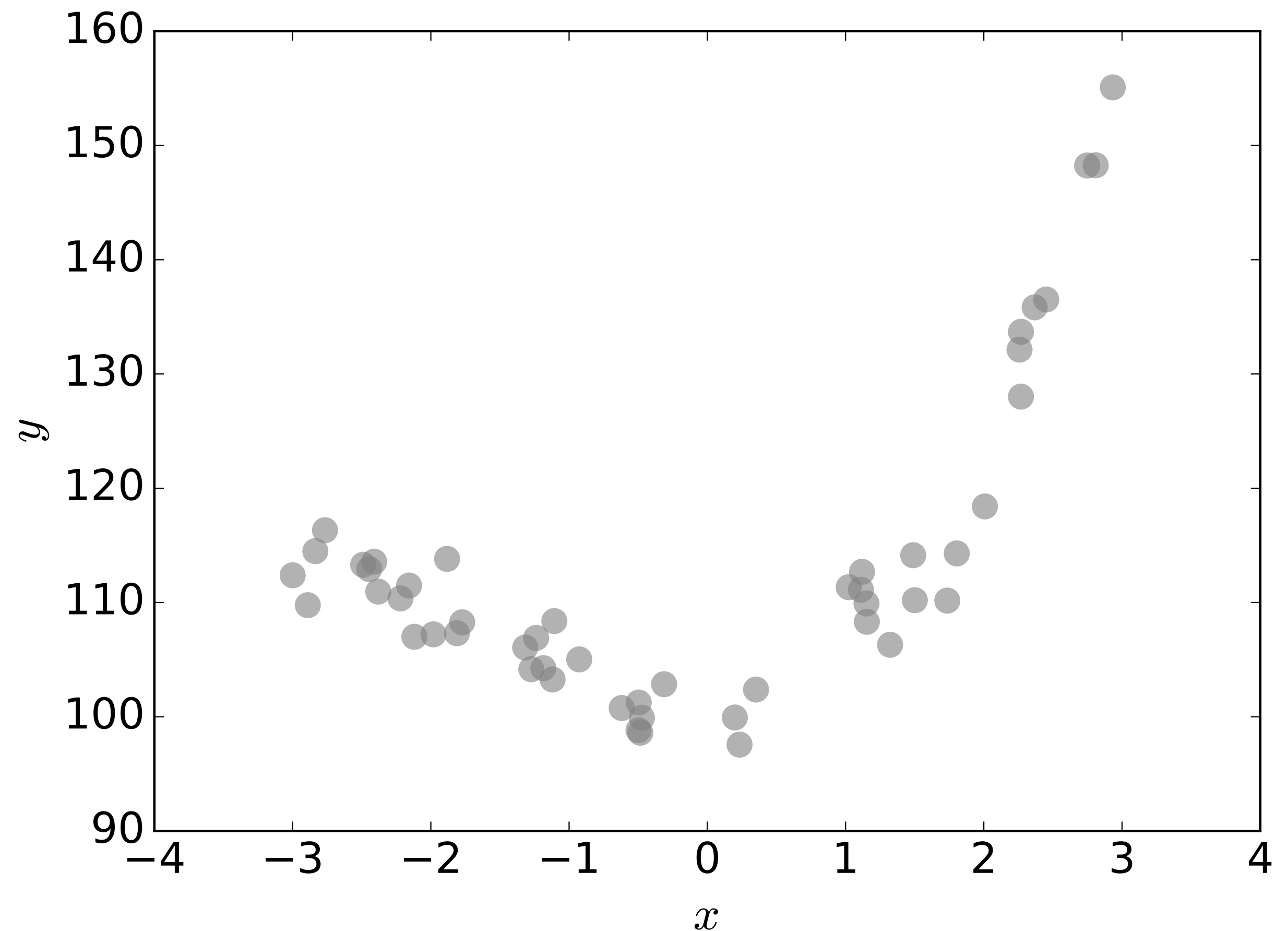
Parameters of the Model

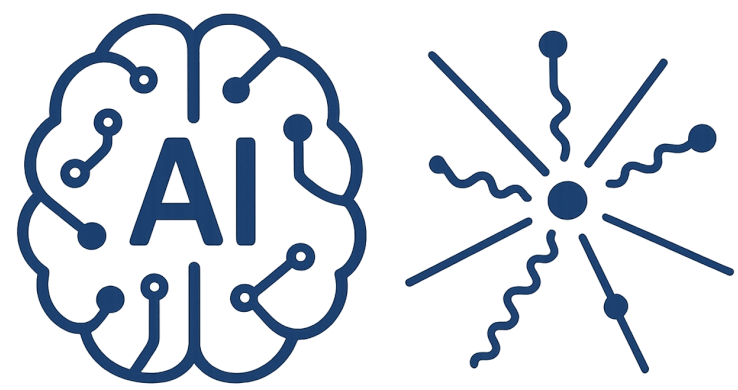
Suppose we have a sample which is drawn from a population

Unstructured data such as intensity as a function of energy (spectrum)

Count minimum number of parameters needs to model the data ?

x	y
x_1	y_1
x_2	y_2
x_3	y_3
x_n	y_n





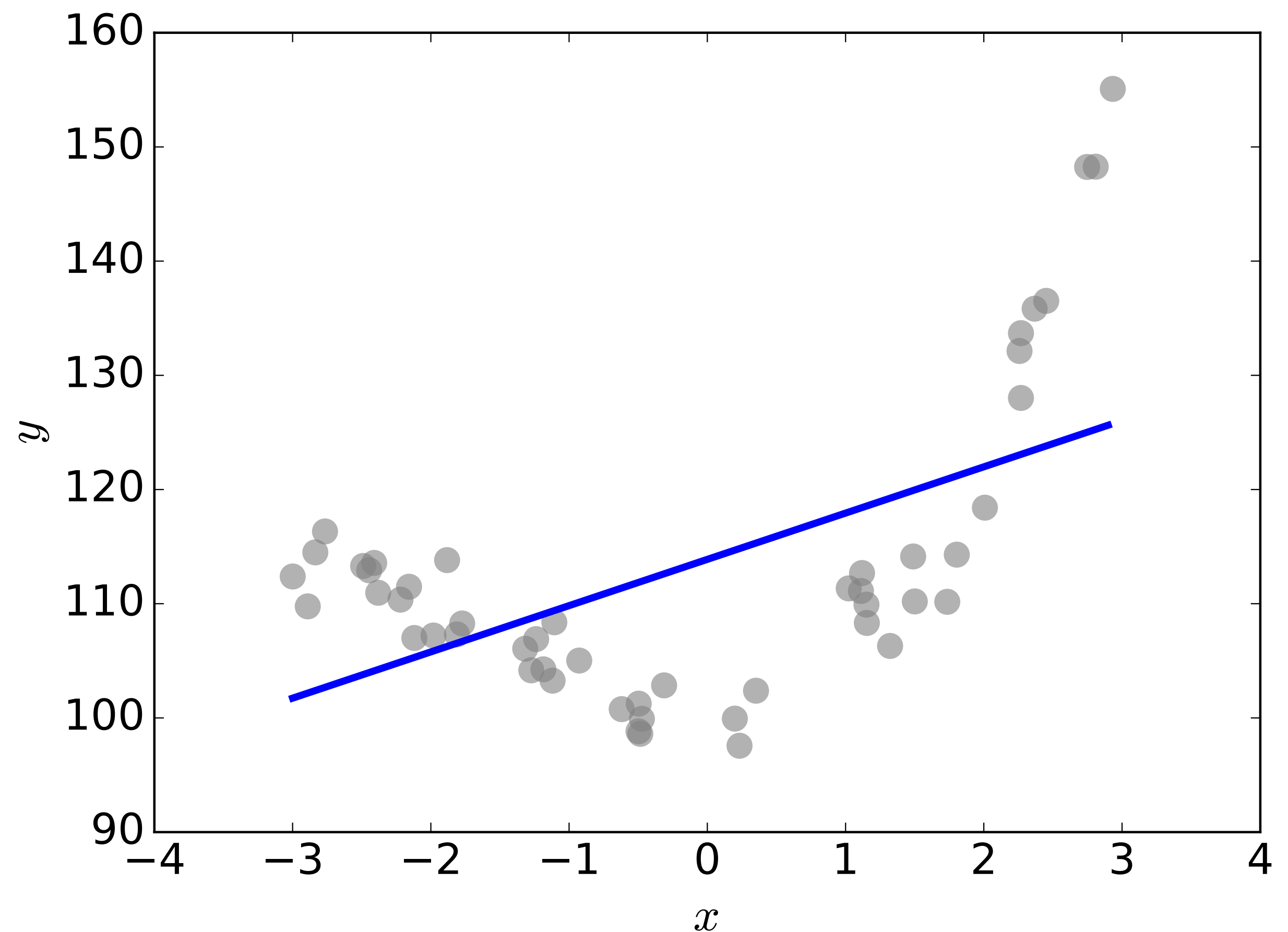
Parameters of the Model

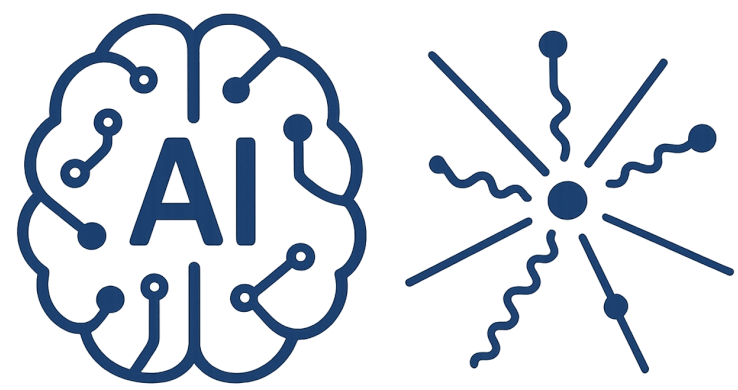
Suppose we have a sample which is drawn from a population

Unstructured data such as intensity as a function of energy (spectrum)

Count minimum number of parameters needs to model the data ? at least two parameters

x	y
x_1	y_1
x_2	y_2
x_3	y_3
x_n	y_n





Parameters of the Model

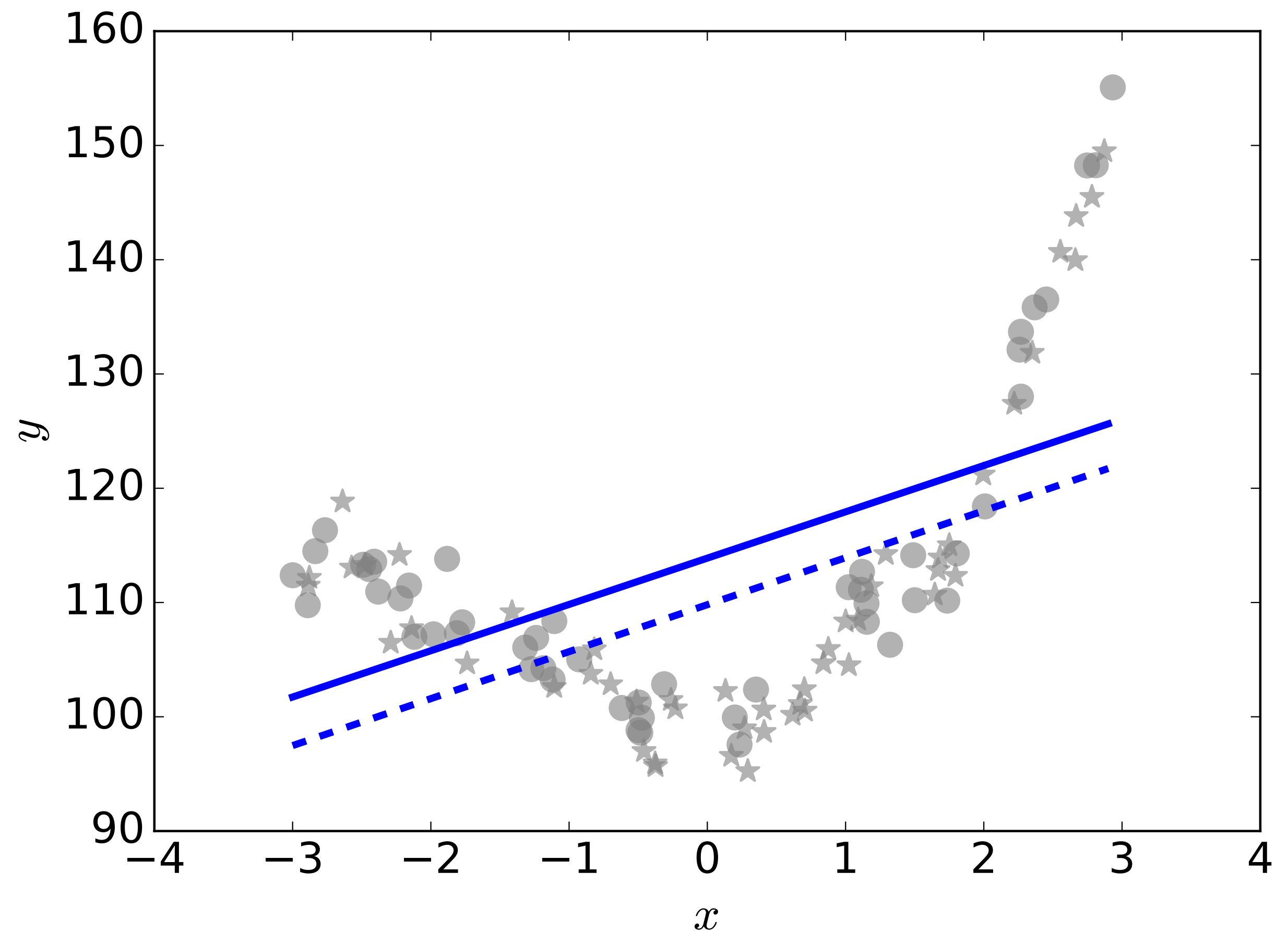
Suppose we have a sample which is drawn from a population

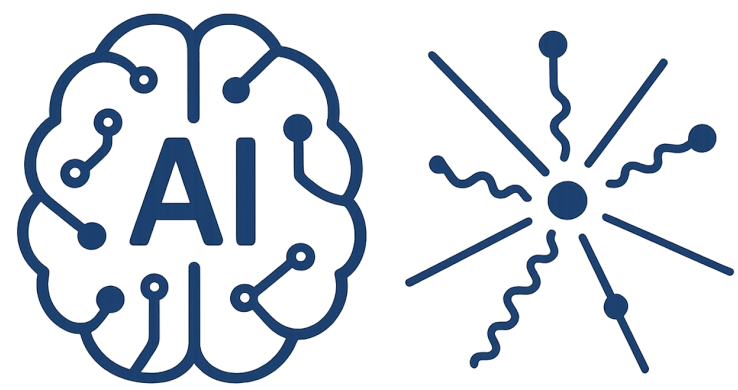
Unstructured data such as intensity as a function of energy (spectrum)

Count minimum number of parameters needs to model the data ? at least two parameters

High bias Low variance

X	Y
x_1	y_1
x_2	y_2
x_3	y_3
x_n	y_n





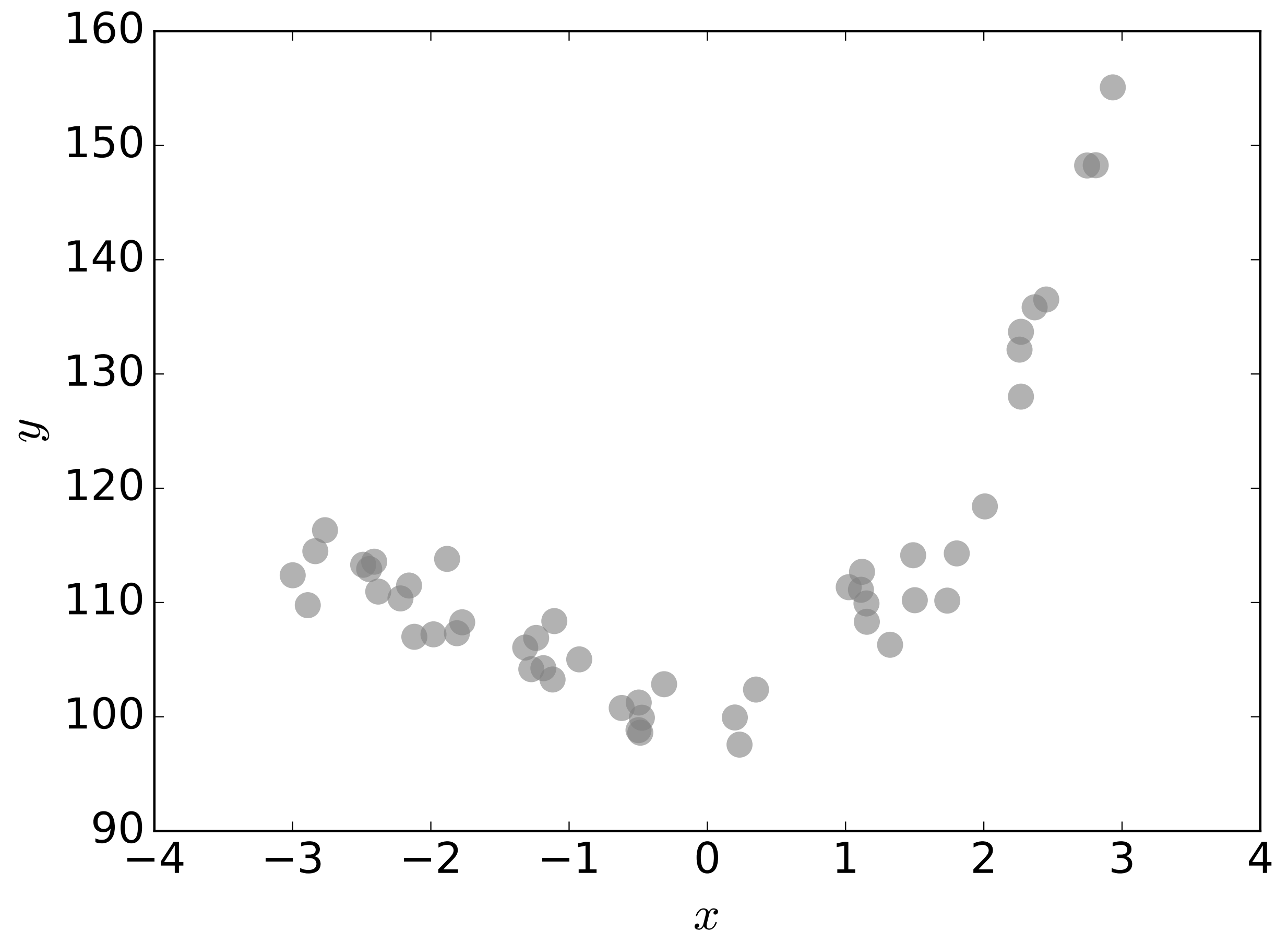
Parameters of the Model

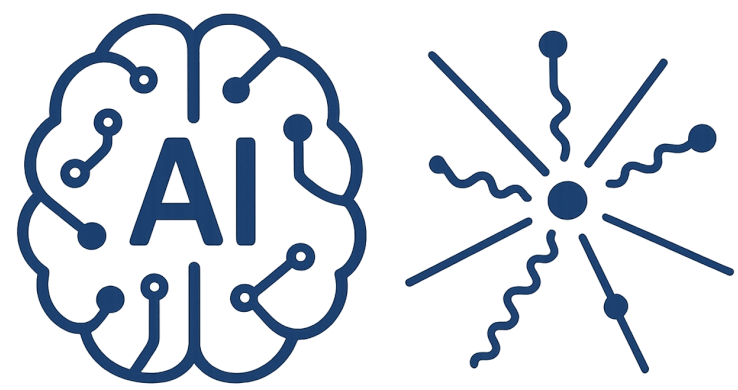
Suppose we have a sample which is drawn from a population

Unstructured data such as intensity as a function of energy (spectrum)

Count maximum number of parameters needs to model the data ?

x	y
x_1	y_1
x_2	y_2
x_3	y_3
x_n	y_n





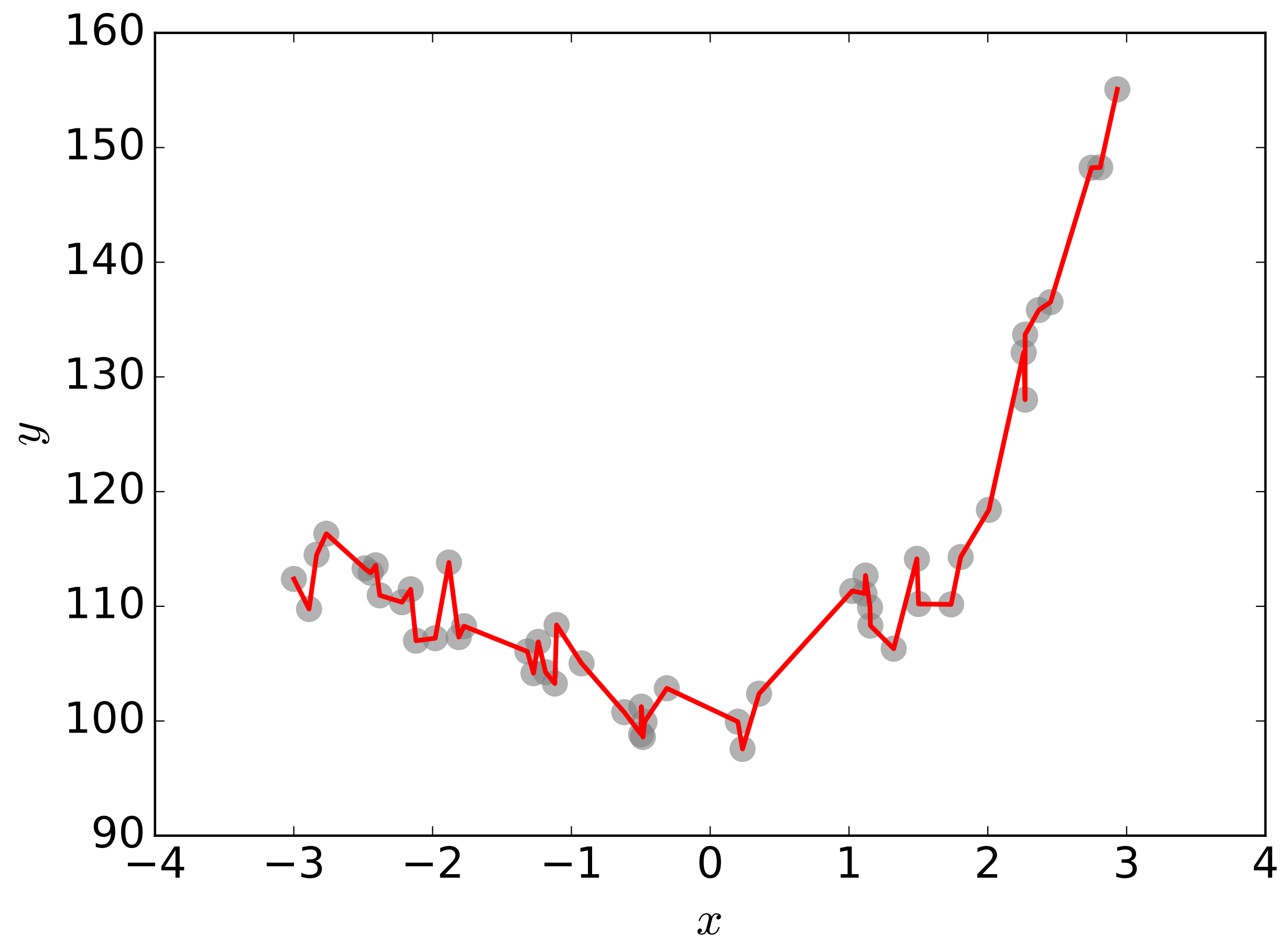
Parameters of the Model

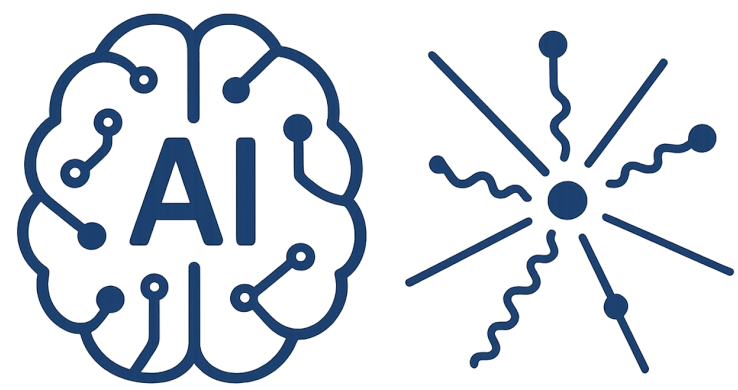
Suppose we have a sample which is drawn from a population

Unstructured data such as intensity as a function of energy (spectrum)

Count maximum number of parameters needs to model the data ? at most $n - 1$ parameters

x	y
x_1	y_1
x_2	y_2
x_3	y_3
x_n	y_n





Parameters of the Model

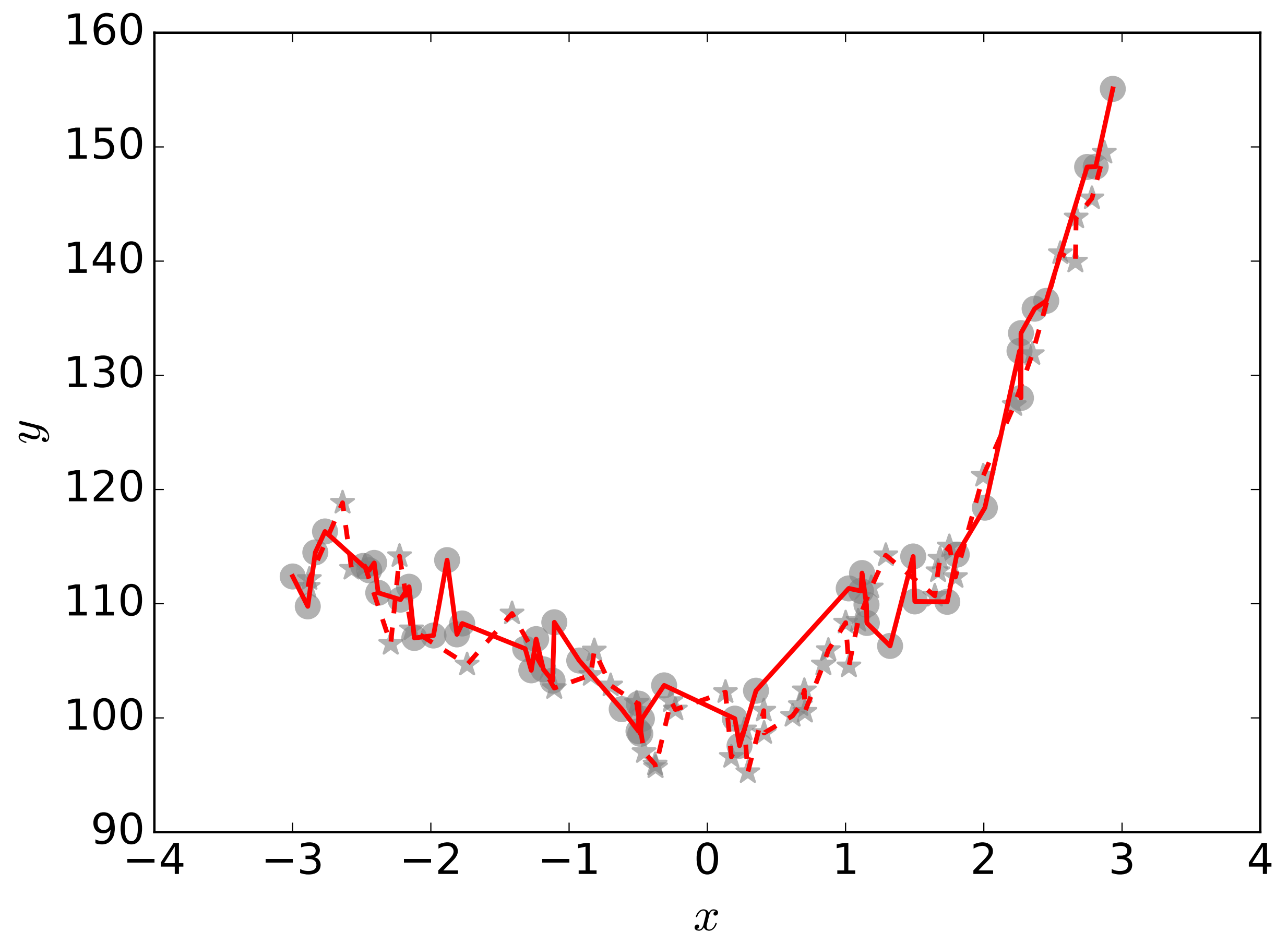
Suppose we have a sample which is drawn from a population

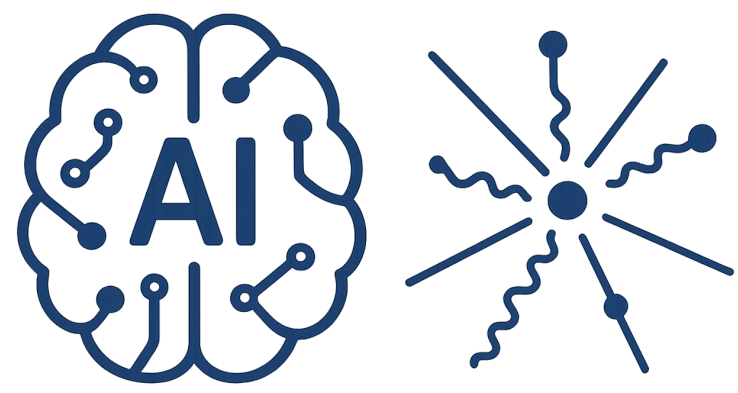
Unstructured data such as intensity as a function of energy (spectrum)

Count maximum number of parameters needs to model the data ? at most $n - 1$ parameters

Low bias High variance

X	Y
x_1	y_1
x_2	y_2
x_3	y_3
x_n	y_n





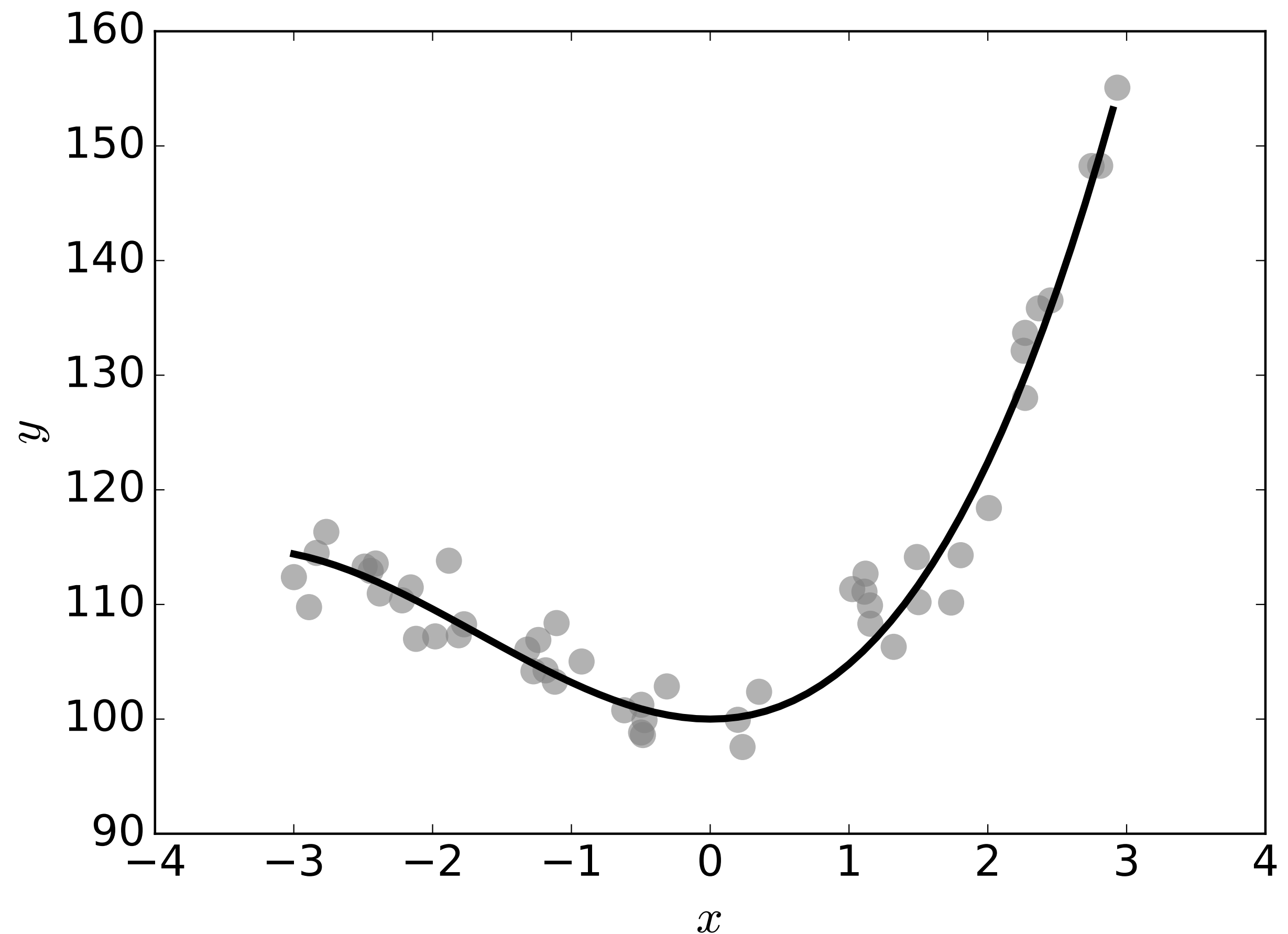
Parameters of the Model

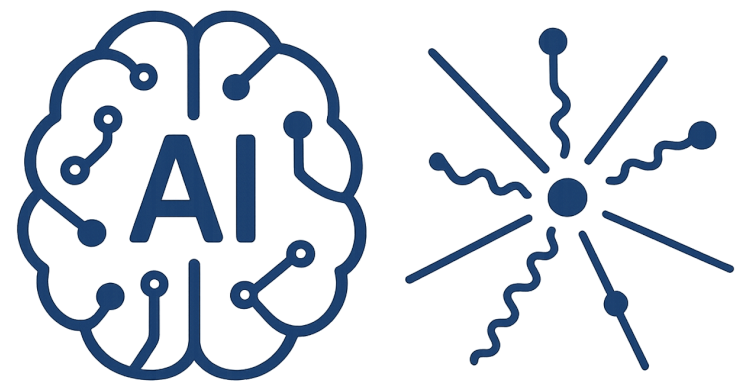
Suppose we have a sample which is drawn from a population

Unstructured data such as intensity as a function of energy (spectrum)

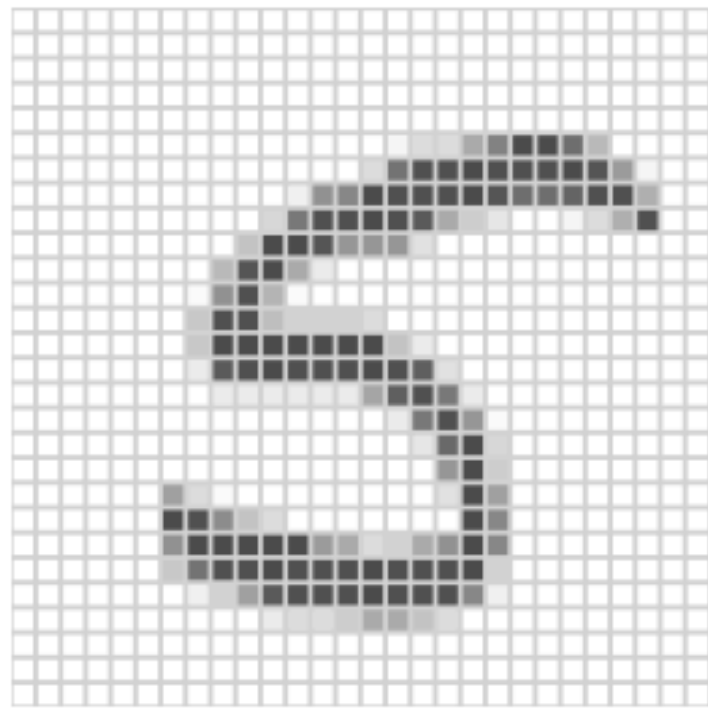
What about the optimum number of parameters ? **5**

x	y
x_1	y_1
x_2	y_2
x_3	y_3
x_n	y_n





Parameters of the Model

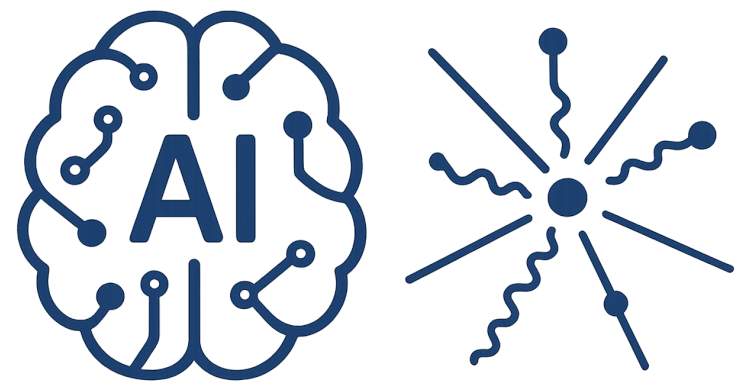


minimum number of parameters
needs to model the data

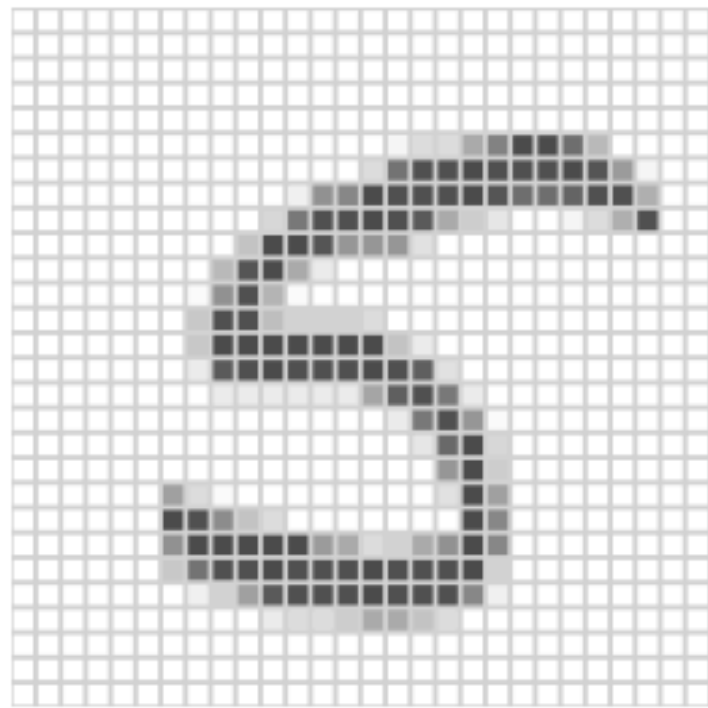
multinomial logistic regression

$$\Pr(Y = k|X = x) = \frac{e^{\beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p}}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1}x_1 + \dots + \beta_{lp}x_p}}$$

~ 7000



Parameters of the Model



minimum number of parameters
needs to model the data

multinomial logistic regression

$$\Pr(Y = k|X = x) = \frac{e^{\beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p}}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1}x_1 + \dots + \beta_{lp}x_p}}$$

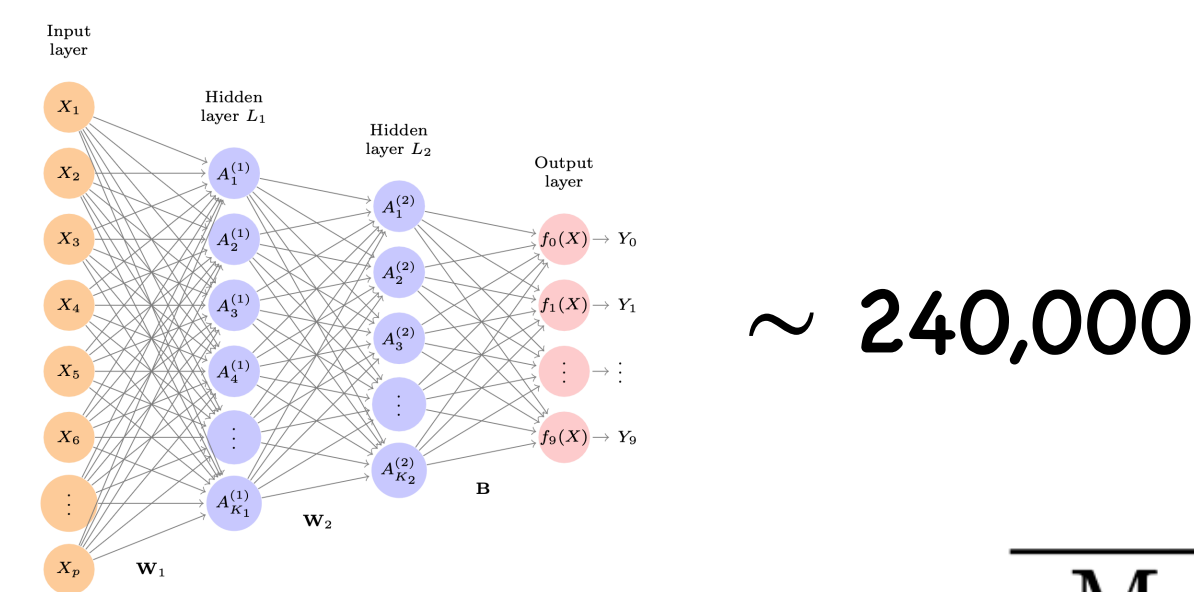
~ 7000

maximum number of parameters
needs to model the data

$$\sim 60,000 \times 7,000 = 420,000,000$$

Parameters of the Model

multinomial logistic regression ~ 7000



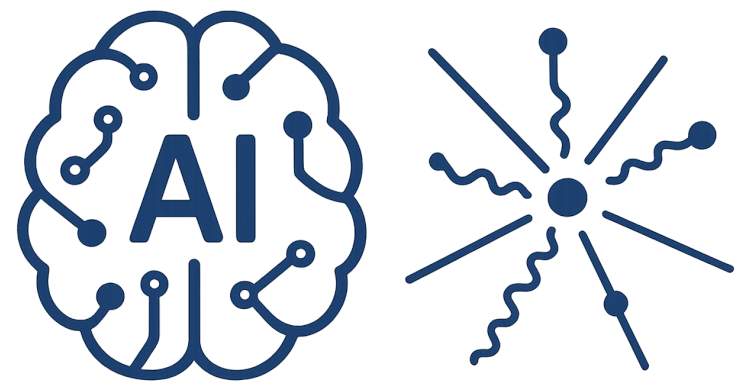
Method	Test Error
Neural Network + Ridge Regularization	2.3%
Neural Network + Dropout Regularization	1.8%
Multinomial Logistic Regression	7.2%

~ 60,000 × 7,000 = 420,000,000

Regularization

Accuracy and Precision

$$\frac{TP + TN}{TP + TN + FP + FN} \quad \frac{TP}{TP + FP}$$



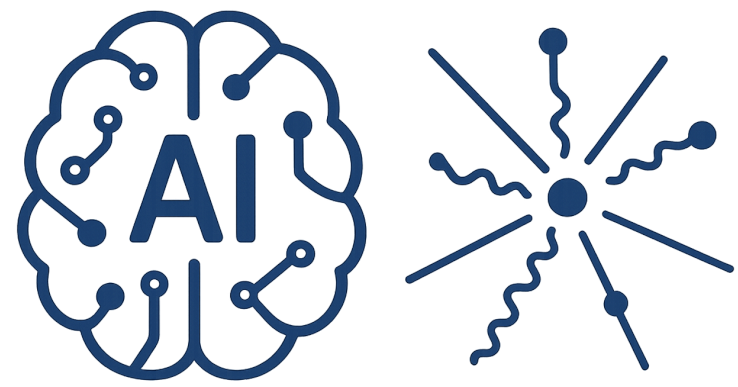
Regularization

Accuracy and Precision

$$\frac{TP + TN}{TP + TN + FP + FN} \quad \frac{TP}{TP + FP}$$

Low Bias

Provided that the true relationship between the response and the predictors is correctly model



Regularization

Accuracy and Precision

$$\frac{TP + TN}{TP + TN + FP + FN} \quad \frac{TP}{TP + FP}$$

Low Bias

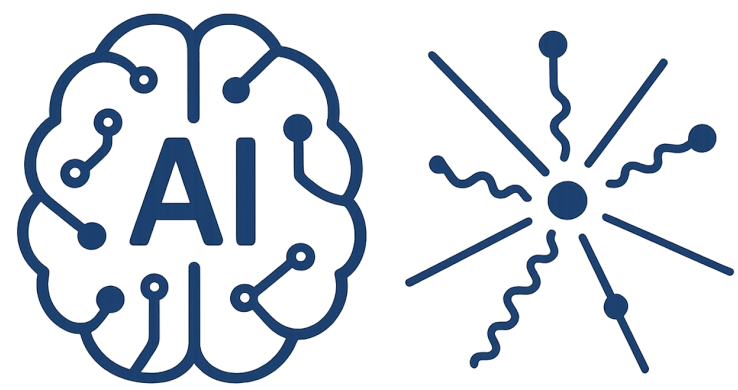
Provided that the true relationship between the response and the predictors is correctly model

$n \gg p$ Low Variance, well test

$n \gtrsim p$ High Variance, poor test

$p > n$ Lots of solution, poor test

$p \gg n$!?



Regularization

Accuracy and Precision

$$\frac{TP + TN}{TP + TN + FP + FN} \quad \frac{TP}{TP + FP}$$

Low Bias

Provided that the true relationship between the response and the predictors is correctly model

$n \gg p$ Low Variance, well test

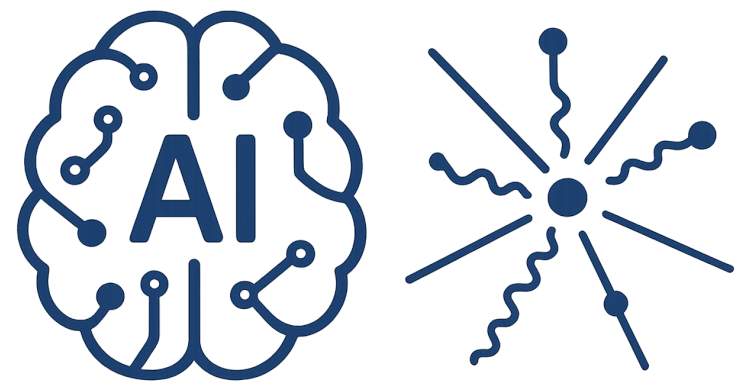
$n \gtrsim p$ High Variance, poor test

$p > n$ Lots of solution, poor test

$p \gg n$!?

Regularization

By constraining or shrinking the estimated parameters, the variance substantially reduce at the cost of a negligible increase in bias



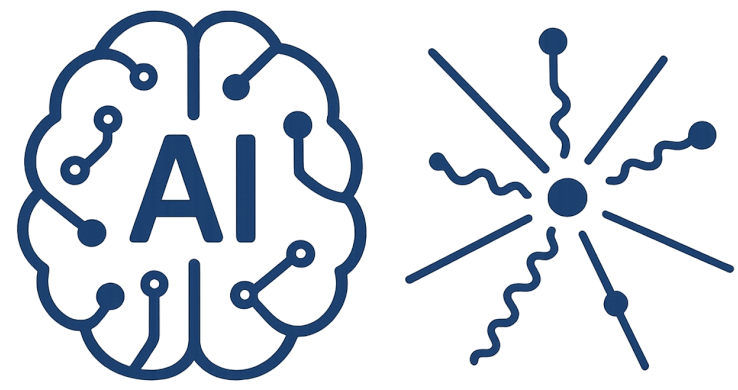
Regularization

Ridge

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

$$\text{RSS} + \lambda \sum_{j=1}^p \beta_j^2$$

the shrinkage penalty is applied to parameters
that control the fluctuation not the intercept



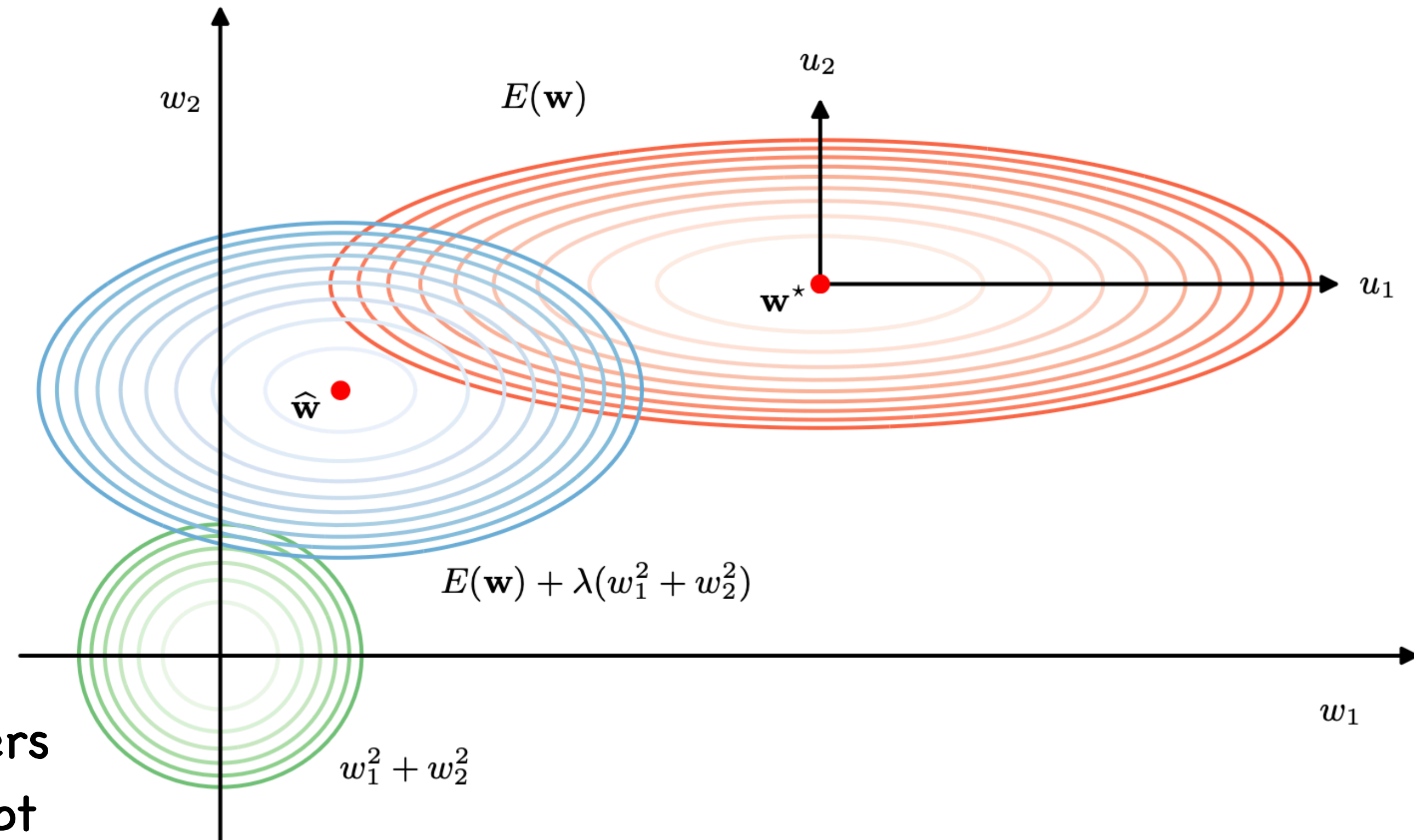
Regularization

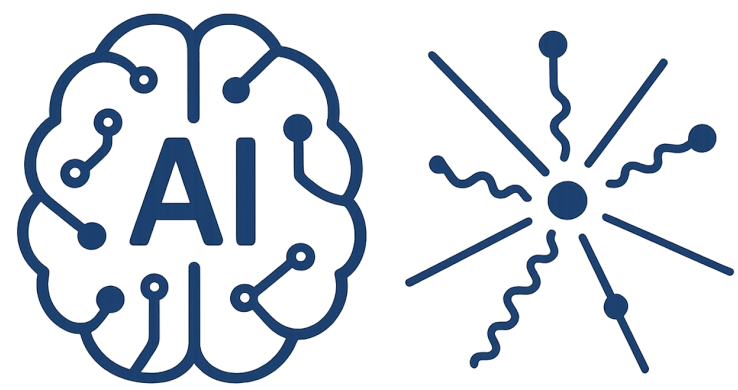
Ridge

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

$$\text{RSS} + \lambda \sum_{j=1}^p \beta_j^2$$

the shrinkage penalty is applied to parameters that control the fluctuation not the intercept





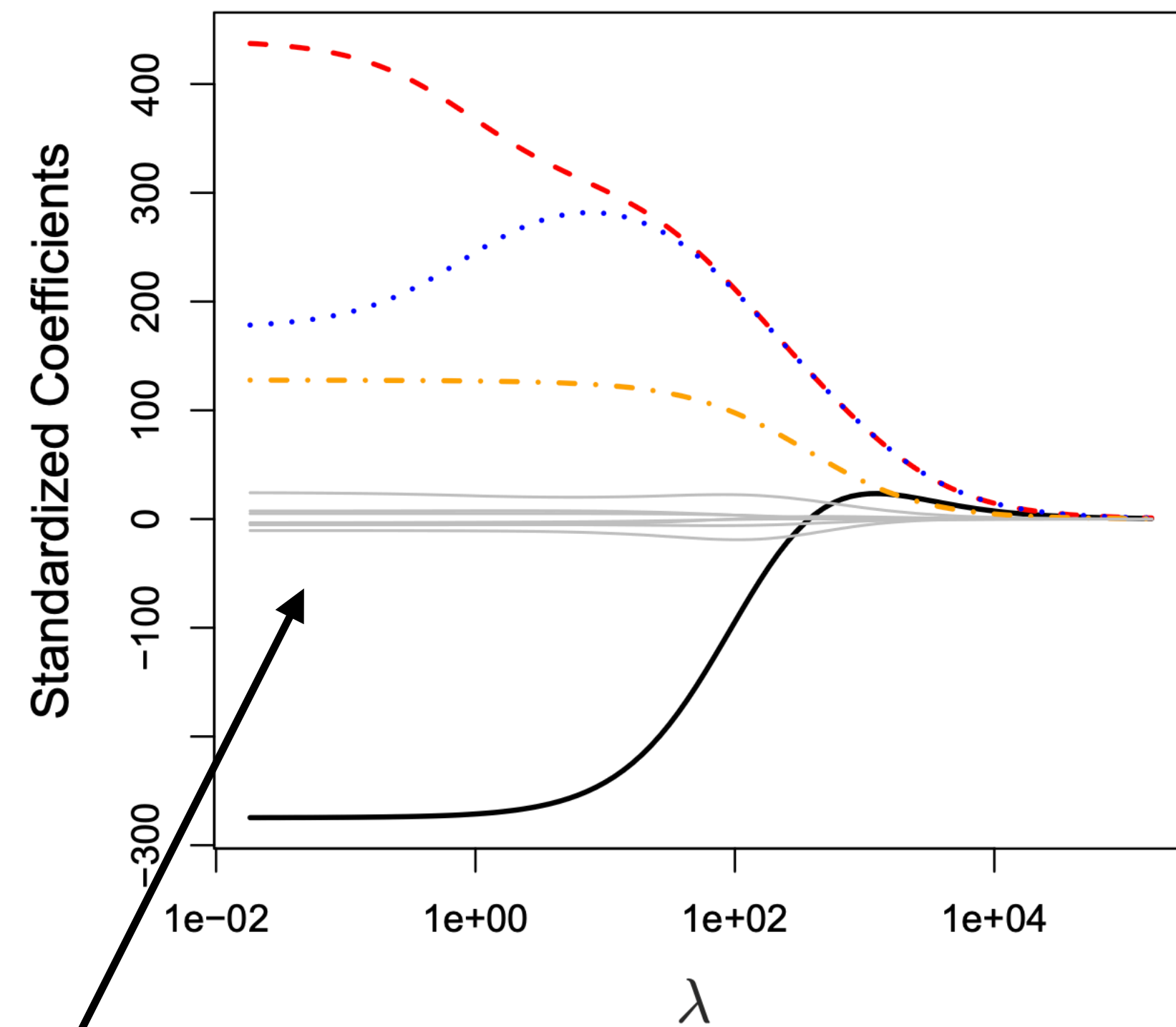
Regularization

Ridge

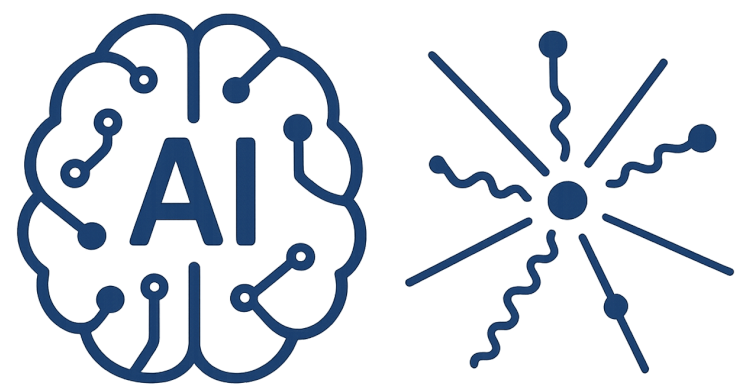
$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

$$\text{RSS} + \lambda \sum_{j=1}^p \beta_j^2$$

the shrinkage penalty is applied to parameters that control the fluctuation not the intercept



Irrelevant parameters



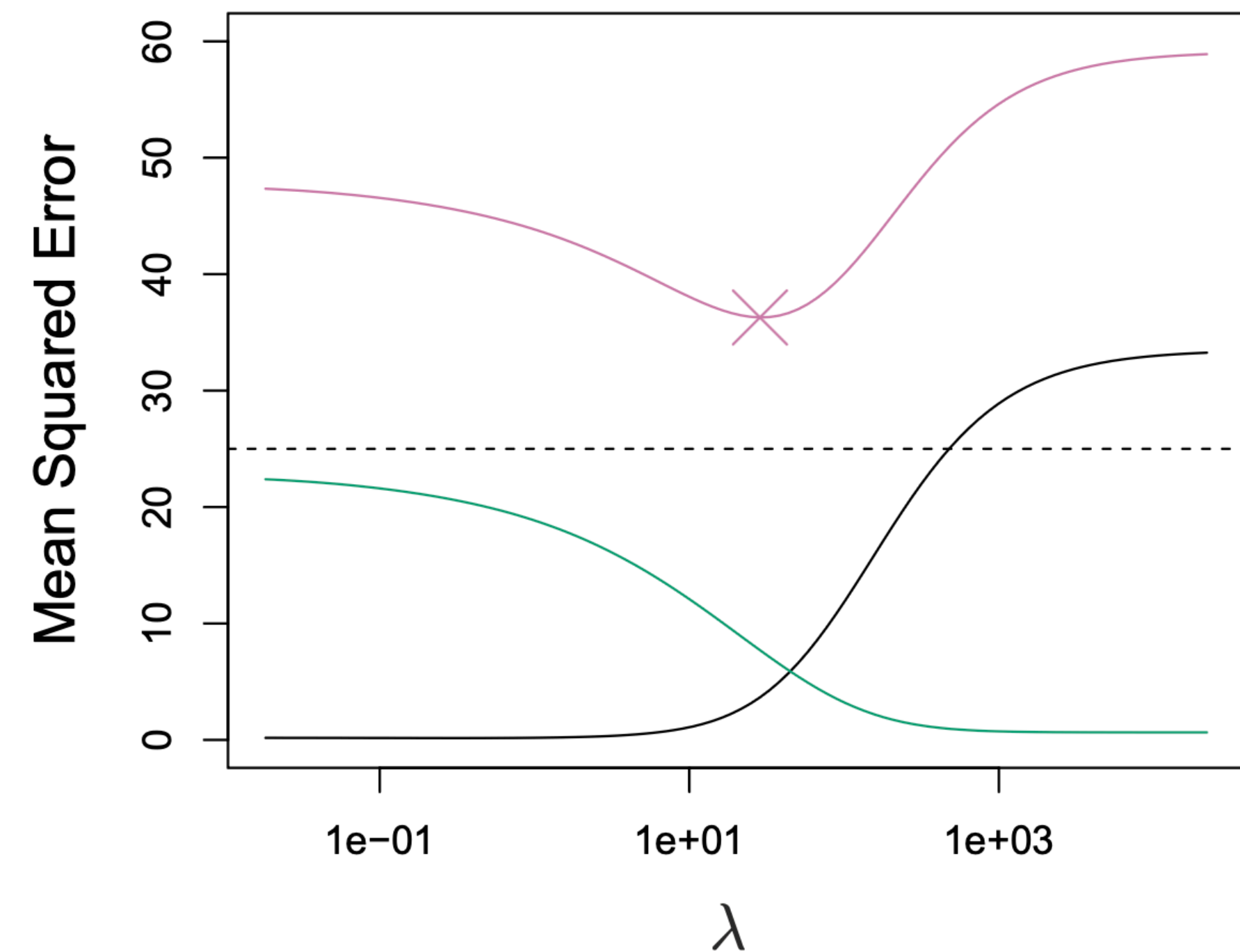
Regularization

Ridge

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

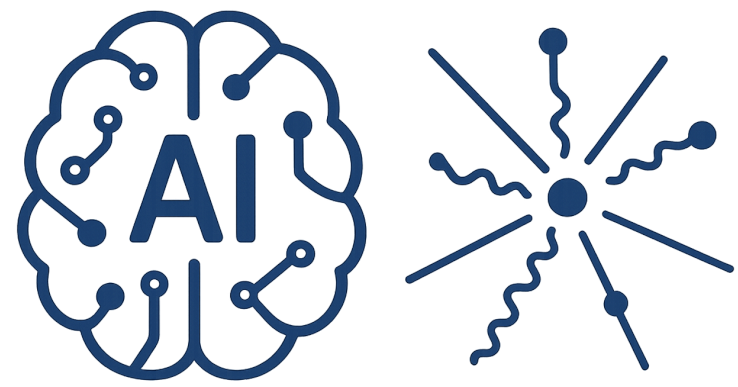
$$\text{RSS} + \lambda \sum_{j=1}^p \beta_j^2$$

the shrinkage penalty is applied to parameters that control the fluctuation not the intercept



$p = 45$, $n = 50$

Variance (green), bias (black), error (purple)



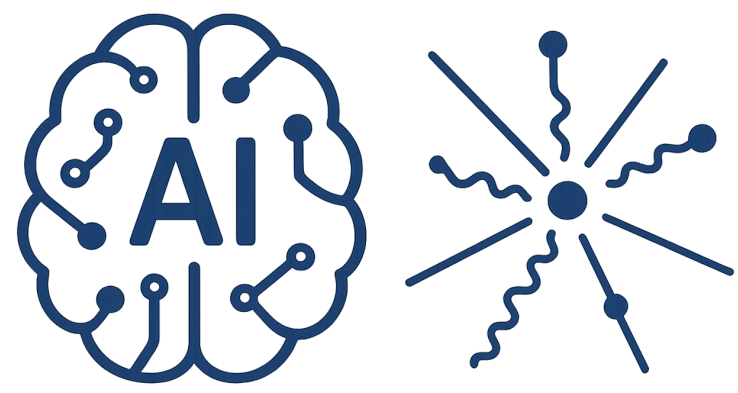
Regularization

Lasso

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

$$\text{RSS} + \lambda \sum_{j=1}^p |\beta_j|$$

the L_1 penalty has the effect of forcing some of the coefficient estimates to be exactly equal to zero when the tuning parameter λ is sufficiently large.



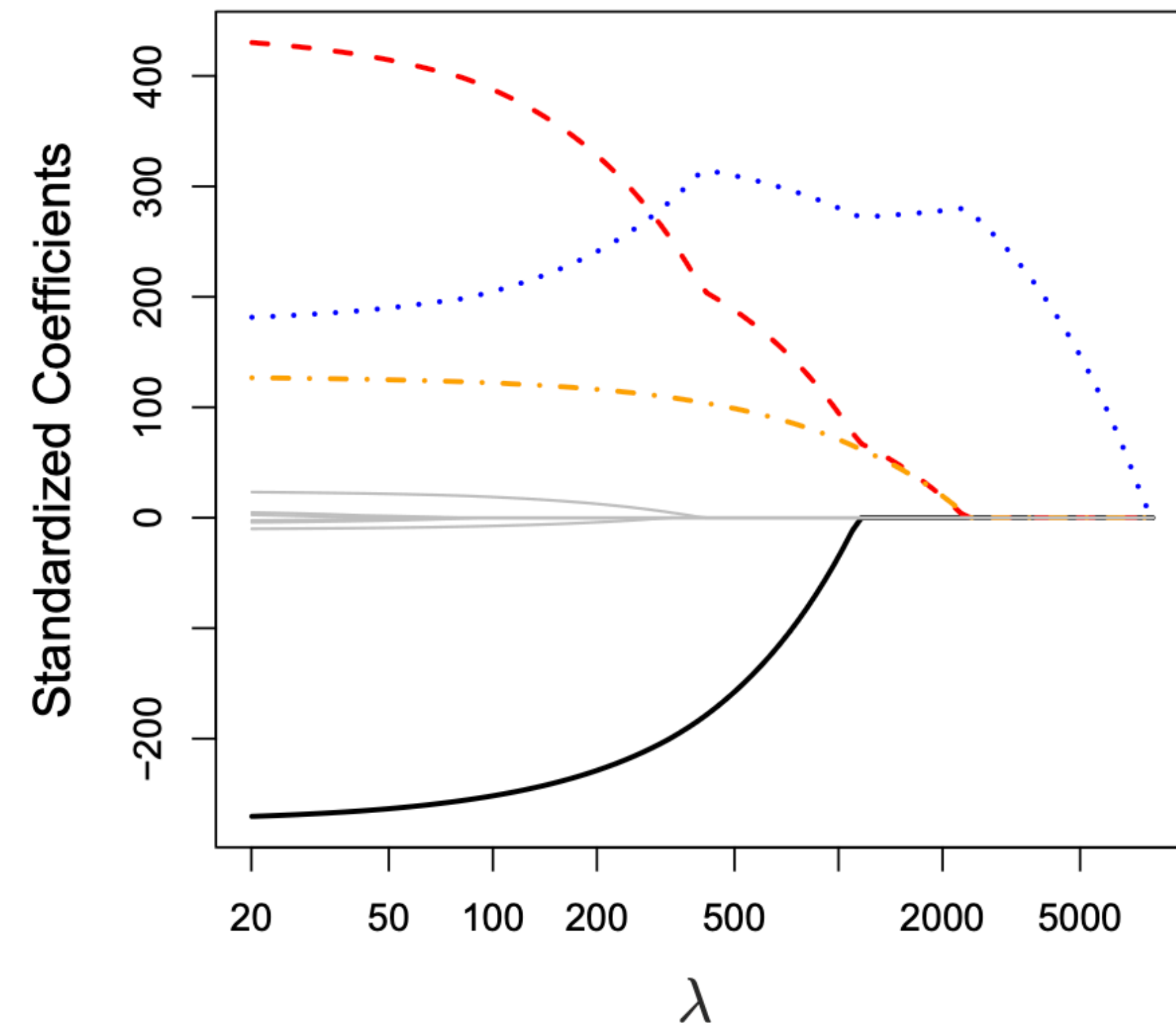
Regularization

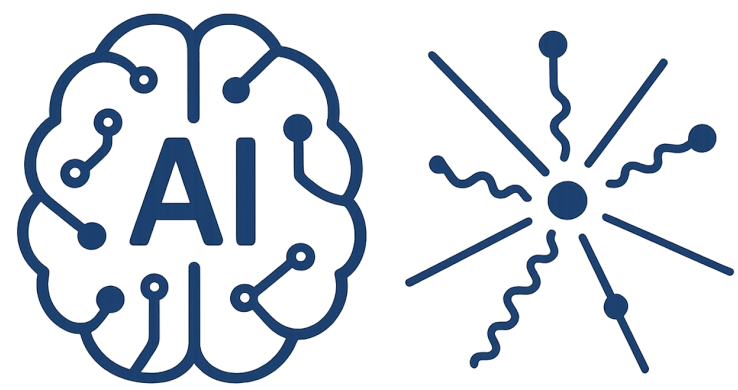
Lasso

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

$$\text{RSS} + \lambda \sum_{j=1}^p |\beta_j|$$

the L_1 penalty has the effect of forcing some of the coefficient estimates to be exactly equal to zero when the tuning parameter λ is sufficiently large.





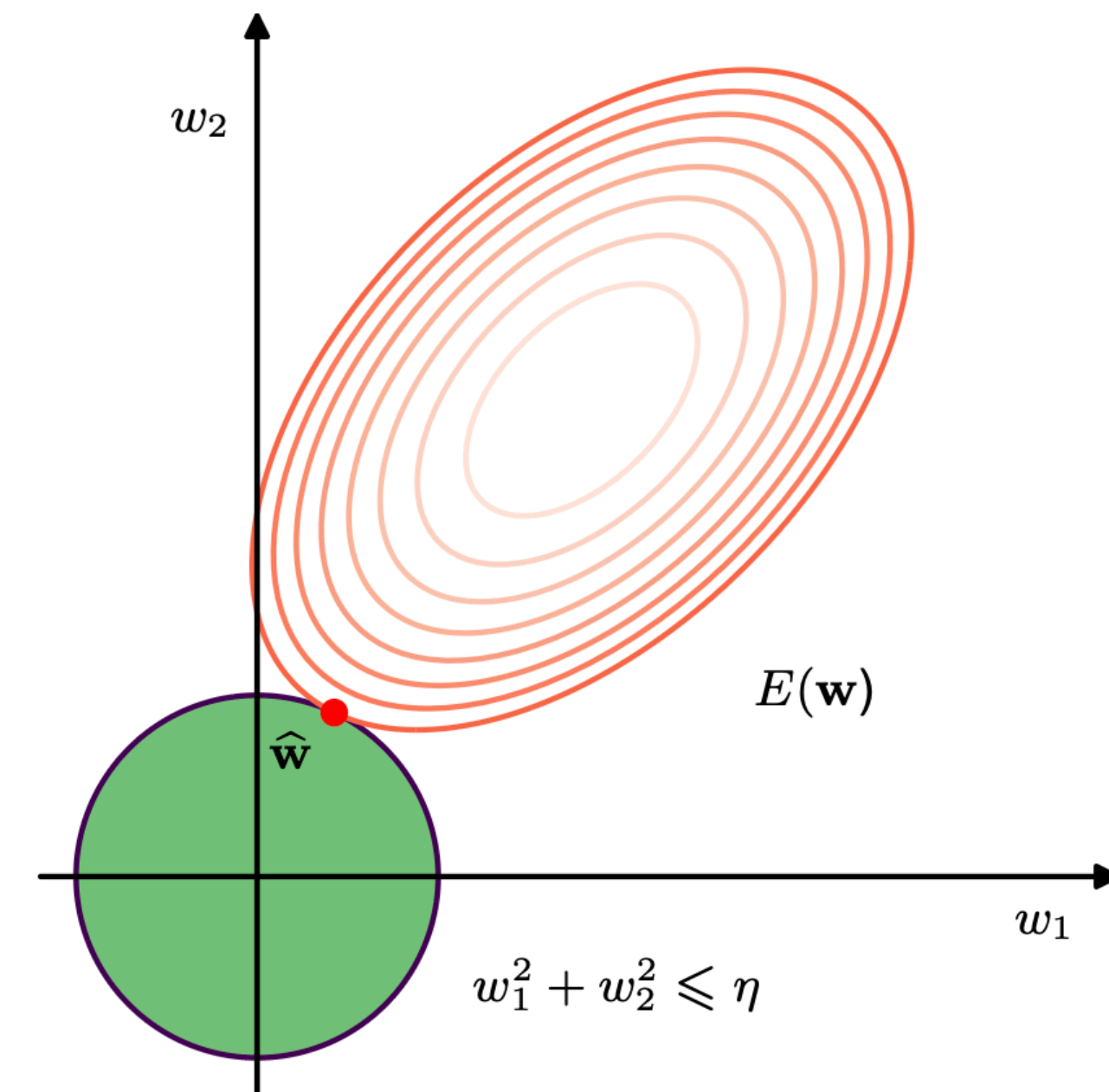
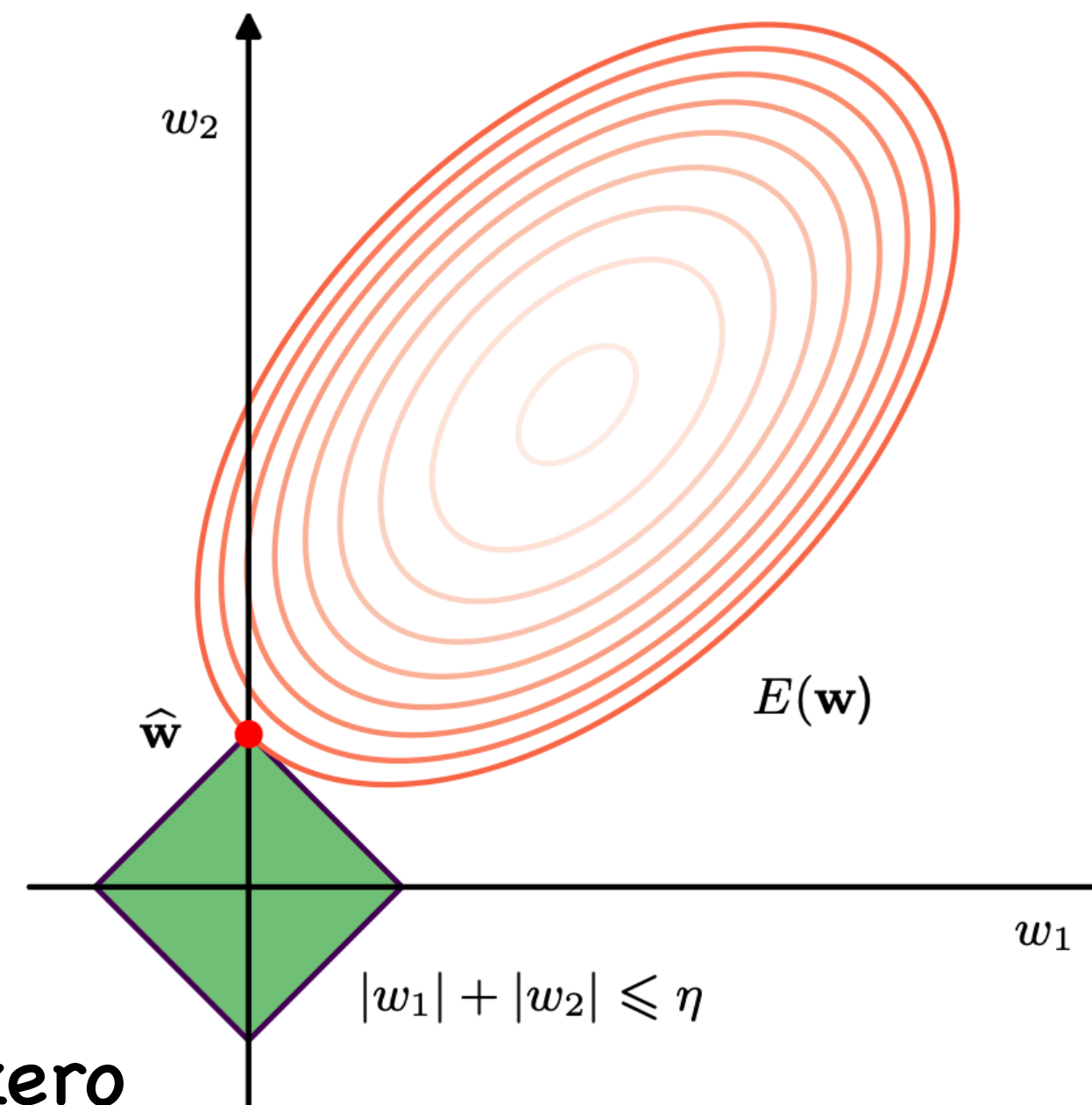
Regularization

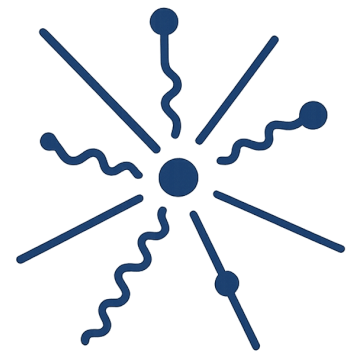
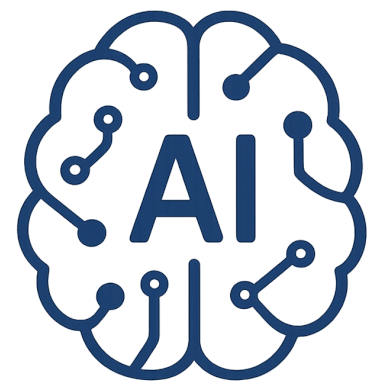
Lasso

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

$$\text{RSS} + \lambda \sum_{j=1}^p |\beta_j|$$

the L_1 penalty has the effect of forcing some of the coefficient estimates to be exactly equal to zero when the tuning parameter λ is sufficiently large.





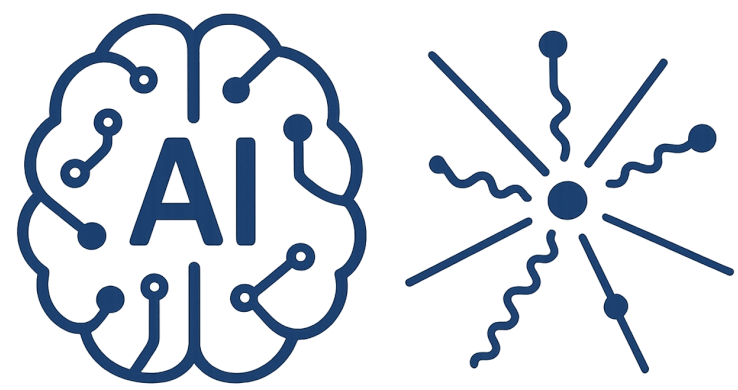
Regularization

It breaks certain desirable transformation properties of network mappings.

$$\text{RSS} + \lambda \sum_j |w_j|^q$$

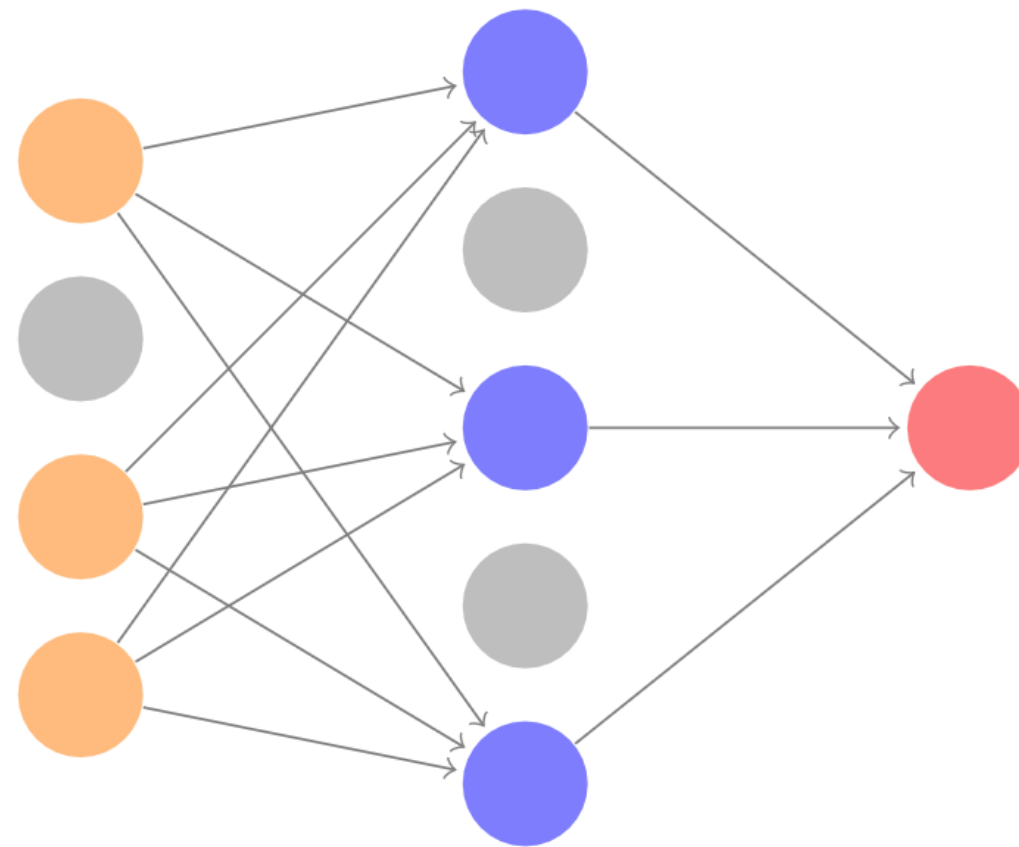
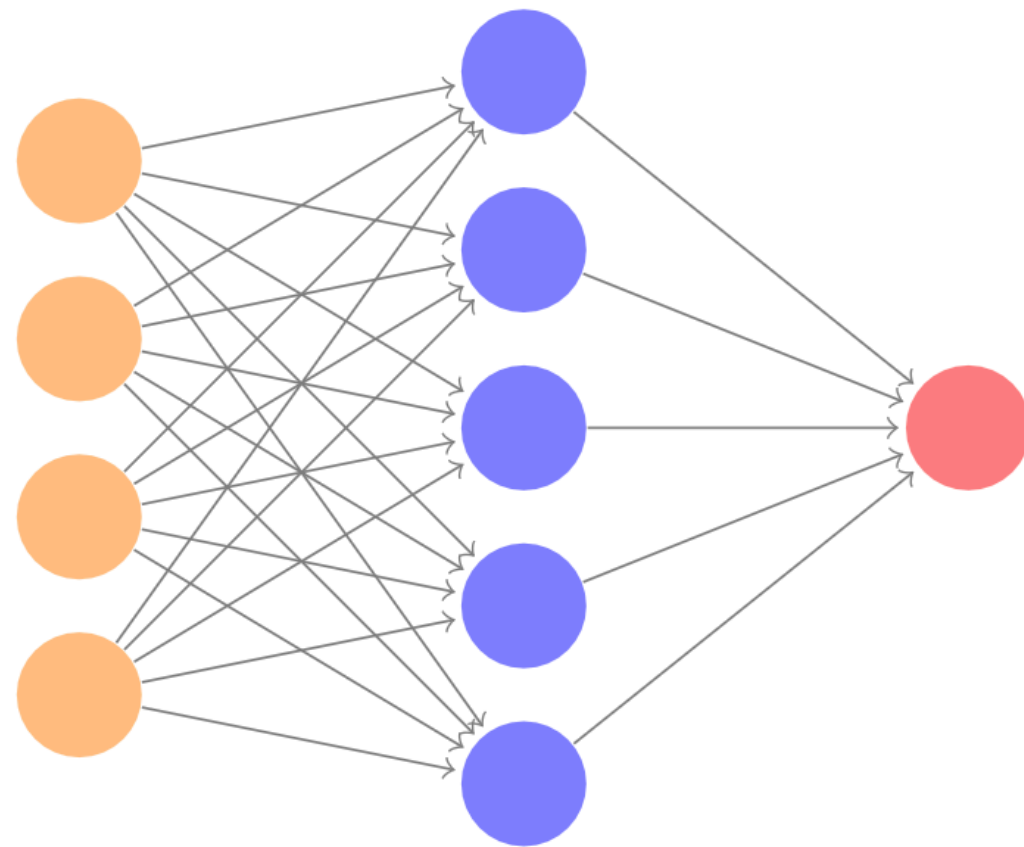
The regularizer should be invariant to rescaling of the weights

$$\frac{\lambda_1}{2} \sum_{w \in \mathcal{W}_1} w^2 + \frac{\lambda_2}{2} \sum_{w \in \mathcal{W}_2} w^2$$



Regularization

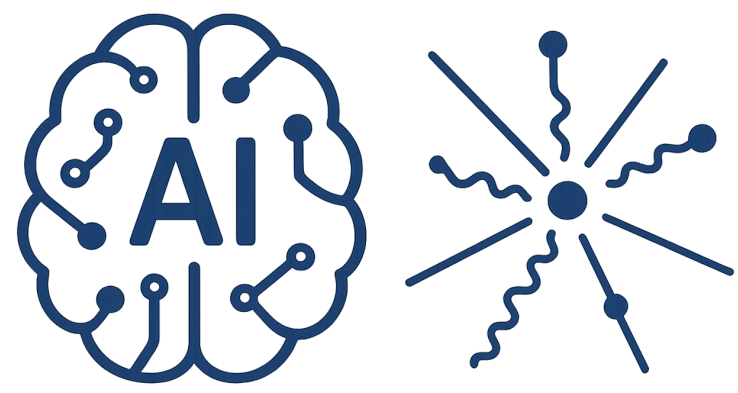
DropOut



remove a fraction of the units in
input layer (0.8) or hidden layers
(0.5) when training

In practice randomly setting the activations for the
“dropped out” units to zero, while keeping the
architecture intact

In the test, rescale the nodes to compensate



Symmetry and invariance

Invariance of the model under transformation

Rotation

Translation

Scale



(a)



(b)



(c)



(d)

Augmentation

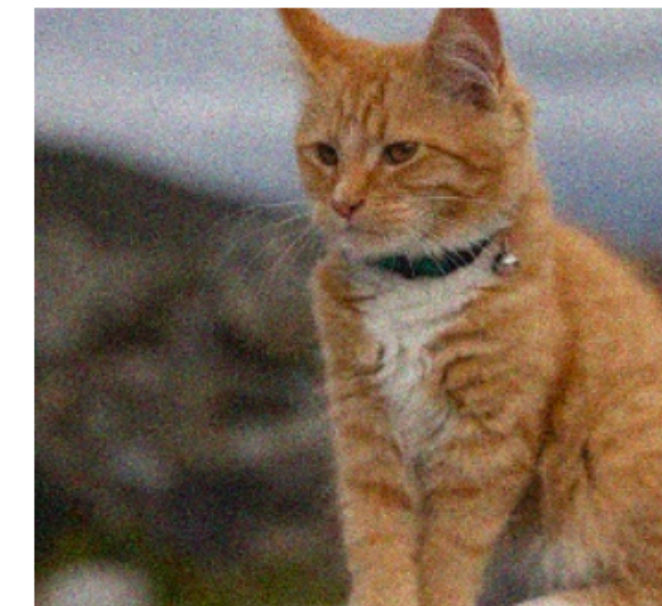
Addition of random noise



(e)



(f)



(g)



(h)